



Topology-aware Federated Learning in Edge Computing: A Comprehensive Survey

JIAJUN WU, Electrical and Software Engineering, University of Calgary, Calgary, Canada

FAN DONG, Electrical and Software Engineering, University of Calgary, Calgary, Canada

HENRY LEUNG, Electrical and Software Engineering, University of Calgary, Calgary, Canada

ZHUANGDI ZHU, Computer Science and Engineering, Michigan State University, East Lansing, United States

JIAYU ZHOU, Computer Science and Engineering, Michigan State University, East Lansing, United States

STEVE DREW, Electrical and Software Engineering, University of Calgary, Calgary, Canada

The ultra-low latency requirements of 5G/6G applications and privacy constraints call for distributed machine learning systems to be deployed at the edge. With its simple yet effective approach, federated learning (FL) is a natural solution for massive user-owned devices in edge computing with distributed and private training data. FL methods based on FedAvg typically follow a naive star topology, ignoring the heterogeneity and hierarchy of the volatile edge computing architectures and topologies in reality. Several other network topologies exist and can address the limitations and bottlenecks of the star topology. This motivates us to survey network topology-related FL solutions. In this paper, we conduct a comprehensive survey of the existing FL works focusing on network topologies. After a brief overview of FL and edge computing networks, we discuss various edge network topologies as well as their advantages and disadvantages. Lastly, we discuss the remaining challenges and future works for applying FL to topology-specific edge networks.

CCS Concepts: • **General and reference** → **Surveys and overviews** • **Computing methodologies** → **Distributed computing methodologies; Machine learning** • **Networks** → **Network architectures** • **Computer systems organization** → **Distributed architectures**;

Additional Key Words and Phrases: Topology-aware federated learning, star topology, tree topology, decentralized topology, hybrid topology, blockchain topology

ACM Reference Format:

Jiajun Wu, Fan Dong, Henry Leung, Zhuangdi Zhu, Jiayu Zhou, and Steve Drew. 2024. Topology-aware Federated Learning in Edge Computing: A Comprehensive Survey. *ACM Comput. Surv.* 56, 10, Article 262 (June 2024), 41 pages. <https://doi.org/10.1145/3659205>

Authors' Contact Information: Jiajun Wu, Electrical and Software Engineering, University of Calgary, Calgary, Alberta, Canada; e-mail: jiajun.wu1@ucalgary.ca; Fan Dong, Electrical and Software Engineering, University of Calgary, Calgary, Alberta, Canada; e-mail: fan.dong@ucalgary.ca; Henry Leung, Electrical and Software Engineering, University of Calgary, Calgary, Alberta, Canada; e-mail: leungh@ucalgary.ca; Zhuangdi Zhu, Computer Science and Engineering, Michigan State University, East Lansing, Michigan, United States; e-mail: zhuzhuan@msu.edu; Jiayu Zhou, Computer Science and Engineering, Michigan State University, East Lansing, Michigan, United States; e-mail: jiayuz@msu.edu; Steve Drew, Electrical and Software Engineering, University of Calgary, Calgary, Alberta, Canada; e-mail: steve.drew@ucalgary.ca.

Authors' Contact Current Information: Zhuangdi Zhu, Department of Cyber Security Engineering, George Mason University, Fairfax, United States; e-mail: zzhu24@gmu.edu.



This work is licensed under a Creative Commons Attribution International 4.0 License.

© 2024 Copyright held by the owner/author(s).

ACM 0360-0300/2024/06-ART262

<https://doi.org/10.1145/3659205>

1 INTRODUCTION

Edge computing has been widely deployed in recent years as a strategy to reduce costly data transfer by bringing computation closer to data sources than conventional cloud computing.

Both academia and industry have seen a surge in research relating to edge computing [70, 112, 115]. This is true specifically in the fields of **Industrial Internet of Things (IIoT)** [96], **connected autonomous vehicles (CAVs)** [72], **augmented reality (AR)** [1], wearable technologies [14], and hybrid architectures and systems combining cloud and edge [131, 134, 136, 143]. Edge computing is prevalent in agriculture, energy, manufacturing, telecommunications, and many other domains. It creates a tremendous amount of data at the edge with heterogeneous data distribution patterns. Distributed data of this scale and variety has created a persistent demand for machine learning to drive decision-making processes at the edge of the network.

Edge computing enables data and computational decentralization. Decentralized devices can collaboratively perform machine learning tasks together forming a cohesive network of nodes [81, 133]. We see the urgency and prospects that distributed learning systems will play a vital role as the bread and butter for edge-based decision-making applications [51, 146]. Back in 2016, the EU imposed restrictions under the **General Data Protection Regulation (GDPR)** [129], which is a regulation in EU law regarding data protection and privacy. Commercial companies in the EU are forbidden from collecting, dealing with, or exchanging user information without their consent. China and the United States are implementing similar legislation as well [146]. As a result, **federated learning (FL)** has emerged as an optimal solution for edge computing without violating privacy legislation. Several companies have shown interest in FL applications, including Amazon [21], Google [31], Webank [33], IBM¹ and ByteDance².

More recently, innovative approaches using foundation models [155], **Vision Transformers (ViT)** [164], and pre-trained models [41, 92] are gaining huge attention. Despite its wide applications, critical challenges of edge computing [72, 102] remain in latency, communication costs, service availability, privacy, and fairness, especially for machine learning tasks at the edge. The low latency limits of communication to the cloud and data privacy requirements have inevitably grounded these distributed machine learning tasks: *the data should never leave the edge*. The deployment of edge-based applications demands learning tasks to run at edge infrastructure instead of the remote cloud [29, 135]. The ubiquitous applications of edge computing in multiple industries [57, 153] justify the strong motivation for the research of distributed machine learning in edge computing.

The topology of the edge network can sometimes be largely overlooked. In this survey, the network topology can be treated both as a challenge and a solution. As a challenge, specific topologies impose certain restraints like extra layers of communication and network structures. Whereas a solution in edge computing, topologies offer new ways to address different bottlenecks such as communication overhead, over-dependent to the central server, and so on. Multiple topology structures exist in current FL works, and each topology brings its benefits and challenges. For example, ring topology [59] is utilized to enhance scalability and accommodate diverse client activities, thereby eliminating the need for a central server. Hosseinalipour et al. [39] propose fog learning, a paradigm that intelligently distributes ML model training across nodes, from edge devices to cloud servers. It enhances FL along three major dimensions: network, heterogeneity, and proximity.

1.1 Scope and Contribution

In this survey, we study the various network topology structures that exist in FL. In Table 1, we compare our survey with existing surveys discussing network topology or FL. There is a significant

¹<https://github.com/IBM/FedMA>

²<https://github.com/bytedance/fedlearner>

Table 1. Existing Surveys that Discuss FL or Network Topology Design

Survey	Year	Focus	Topology	FL
Rajaraman [105]	2002	Topology and routing in Ad-hoc Network	✓	×
Li et al. [60]	2006	Overview of topology control techniques	✓	×
Donnet and Friedman [23]	2007	Measurements of network topology	✓	×
Lim et al. [68]	2020	FL in Mobile Edge Networks	×	✓
Kairouz et al. [46]	2021	FL Advances and Open Problems	×	✓
Nguyen et al. [91]	2022	FL for Smart Healthcare domains	×	✓
Nguyen et al. [90]	2023	FL Applications for IoT networks	×	✓
Zhu et al. [165]	2023	Blockchain-empowered FL	×	✓
Ours	2024	Edge Network Topology for FL	✓	✓

gap in the number of surveys conducted on network topology at present. Since the introduction of FL in 2016, we have seen a huge increase in FL-related papers. Many existing surveys look to treat network topology as a system limitation or challenge, while some papers propose to use new network topology to improve communication or computing efficiency. Several comprehensive surveys have extensively covered the general concepts, architectures, and applications in FL [46, 63, 68, 146]. More recently, many studies tend to concentrate on specialized areas within the FL domain, possibly due to the rapid rate at which FL works are being published. Some of the recent work includes FL in the health domain [91], FL for the internet of things [90], and blockchain-empowered FL [165]. However, no existing surveys have discussed or organized research works of FL in edge computing from the network topology perspective. This gap leaves us with a vast open area to discuss FL from a new perspective. Our study examines surveys from a wide range of dates of publication that discussed FL or network topology designs. We found that these two topics were never discussed together, even though FL comprises many different network topologies. This motivated us to showed them here in Table 1 about their difference to us. To our knowledge, no existing surveys have reviewed unique FL works from the edge network topology perspective and have promoted the development of various topology structures in FL. Compared with previous surveys, this paper’s main contributions are:

- (1) Our survey introduces a novel perspective by employing edge network topologies (the network’s structure) to categorize unique FL works.
- (2) We provide a comprehensive classification of FL into four major topologies, including star topologies, mesh topologies, hybrid topologies, and less common network topologies, which provide clarifications for future research.
- (3) We follow a systematic review approach using PRISMA [86] for the paper selection process.
- (4) We present the design, baselines, and benchmarks and then thoroughly review the key findings of some highlighted work.
- (5) We outline promising research directions and challenges for the future development of topology-aware FLs.

The rest of the paper is organized as follows. In Section 2, we explain our research methodology. In Section 3, we introduce an overview of FL in edge computing. In Section 4, we propose eight types of FL network topologies and summarize existing studies along each topology. In Section 5, we present some of the open Issues in edge FL Topology. In Section 5, we explain the limitations and synthesize a roadmap for future research. Last, we conclude our paper in Section 6.

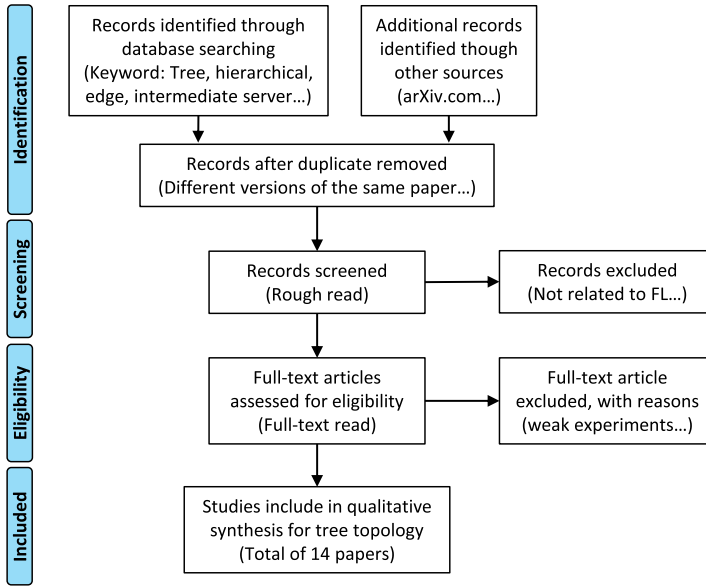


Fig. 1. The PRISMA flow diagram with reasoning for tree topology.

2 RESEARCH METHODOLOGY

2.1 Research Goals Formulation

We aim to provide an in-depth and systematic overview of all papers on FL that utilize one or more unique network topologies. Furthermore, we use the PRISMA [86] search strategy to collect all the papers following a similar approach as Pfitzner et al. [97]. We show an example of searching for tree topology papers using the PRISMA flow diagram in Figure 1. Additionally, we aim to show evidence that different network topologies and FL can benefit from each other. We summarize our research goals into three points.

- Identify existing edge network topology structures in the current FL literature.
- Examine the unique challenges and benefits different topologies bring.
- Provide readers with an overview of the baseline methods and datasets used in each paper.

2.2 Search Strategy

For our paper search strategy, we start by searching papers that contain the terms “*federated learning AND topology*.” We submitted this search criteria to the digital scholarly databases. The three main scholarly databases we used are the digital library of the Association for Computing Machinery, the online portal of the Institute of Electrical and Electronics Engineers, and Google Scholar. This search strategy only returned a few papers and it was not helpful. Therefore, we modified our search strategy to treat every topology as its own branch of work and restarted the search process. The detailed search process is listed below:

- (1) Identity a topology structure to start the search process (star, tree, ... or mesh topology).
- (2) Initailite the PRISMA [86] search process.
- (3) Group the specific topology paper into major topology or minor topology.

2.3 Inclusion and Exclusion Criteria

Our survey aims to give readers a good understanding of FL, and the upsides and downsides of several network topology structures, so we have selected the following criteria for

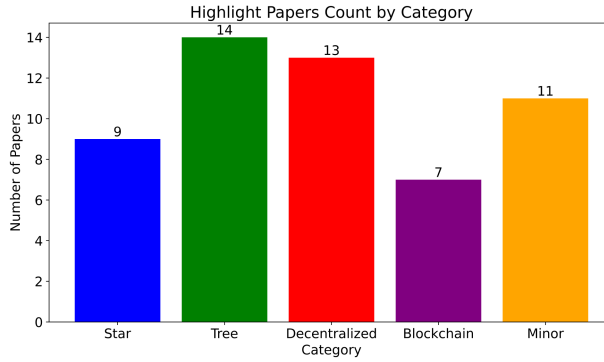


Fig. 2. Number of papers for each network topology type.

inclusion. After collecting the papers returned from the database search, we include papers that are:

- Peer-reviewed (Identification phase).
- Presenting one or more unique topology structures (Identification phase).
- Using FL as the primary methodology (Screening phase).
- Implanting and comparing the proposed method with strong baseline methods (Eligibility phase).

However, the surveyed query terms return many irrelevant works to this review. Some papers may contain only one or two mentions of FL and cover completely unrelated topics. Our exclusion criteria are listed below:

- Share different titles but are different versions of the same paper (Identification phase).
- FL is not involved at all (Screening phase).
- Experiments do not use known baseline methods (Eligibility phase).
- Application-focused or case study (Eligibility phase).
- Benchmark paper evaluating existing works (Eligibility phase).

We use our proposed strategy to search for eight topologies in Section 2.2. We organize the results into five major topology types shown in Figure 5. A total of 42 papers meet all selection criteria. We also illustrate the number of papers from each topology in Figure 2.

3 AN OVERVIEW OF FEDERATED LEARNING IN EDGE COMPUTING

3.1 Background

Recent years have embraced effervescent advances of Federated Learning (FL) algorithms in various applications, including IoT [85], healthcare [107], image processing [51], and the like.

In the representative federated learning approach *FedAvg* [83], with the restrictions of GDPR [129], each mobile device learns a local model and updates the model to a central server periodically. A central server then aggregates local models using a simple yet effective method to produce a global model and distributes the global model to all Android devices for the next learning cycle. The FedAvg algorithm improves over FedSGD, which uses parallel **stochastic gradient descent (SGD)**. FedSGD selects a set of workers each round, and the selected workers compute the gradient using the global model parameters and their local data. Gradients from each worker are sent back to the server, which performs SGD using the combined gradients and the learning rate. The process is repeated until the model converges. Compared to the baseline algorithm FedSGD,

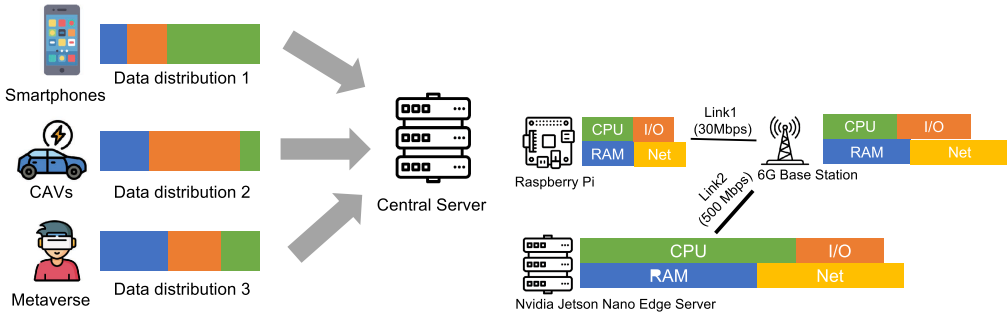


Fig. 3. **Right:** An example showing the statistical heterogeneity among different types of clients in FL. Depending on how the clients generate data, the statistical distributions and patterns of data on each device can be very different. **Left:** A demonstration of system heterogeneity. Three different tiers of edge devices have distinct capabilities of computing, connected by links with different bandwidths.

FedAvg requires significantly fewer rounds of communication to converge. As with FedSGD, FedAvg followed the general computation steps where the server sends the model parameters to each worker. Each worker computes the gradient using the received model parameters, its local data, and a given learning rate. FedAvg differs from FedSGD in that each worker repeats the training process multiple times before sending the updated model parameters back to the server. FedAvg was developed with the intention of achieving the same level of efficacy with less communication to the server. While the overall computation task for each worker increased, there were fewer rounds of communication compared to FedSGD, resulting in a trade-off between computation and communication costs. In many FL scenarios, the edge clients generally have limited data residing locally [5, 83]. Even though deep models are commonly used, the computational expenses are often overshadowed by the communication costs. This is why FL with FedAvg algorithm [83], known for its communication efficiency, is particularly effective.

We categorize the FL algorithms we surveyed based on their emphasis on the types of challenges they tackled.

3.1.1 Statistical and System Heterogeneity. A significant amount of effort has been made to address the issue of *user heterogeneity* in FL. Specifically, the heterogeneity is manifested in both *statistically* and *systematically*.

On the one hand, *statistical heterogeneity* in FL refers to the differences among user local data distributions as shown in Figure 3. Namely, the clients have datasets that are **not independent and identically distributed (Non-IID)**. When the data is collected locally, such differences are likely induced by heterogeneous user behavior. In the case of Non-IID data distributions, aggregation may lead to a biased global model with a sub-optimal generalization performance. This phenomenon is also known as client-drift [47], and it refers to the process by which global models are updated toward local optimal solutions as a result of heterogeneous data.

Towards addressing this client-drift issue, previous works including FedProx [111], pFedMe [123], and SCAFFOLD [47], have proposed constraining the local model parameters to prevent them from diverging far from the global model optimal. *Personalized FL* is an alternative strategy for handling data heterogeneity. It permits different model parameters or even architectures to be adopted by local users. Besides diversified architectures, few-shot adaptations can also achieve personalization by fine-tuning a global model using local data [62, 123].

In the meantime, *system heterogeneity* results from different user capacities in terms of computation, memory, bandwidth, and so on. We show an example of system heterogeneity in Figure 3. Adopting one unified model architecture for FL can be undesirable under such scenarios: an

over-large global model might bring heavy workloads to small users which lack computation or transmission resources, while an over-small global model may under-perform in capturing complex feature representations for the learning tasks. Therefore, an emerging group of algorithms is pursuing FL frameworks that support heterogeneous user model architectures [39, 79, 111, 132, 152, 156, 157].

3.1.2 Privacy. Although FL allows decentralized devices to participate in machine learning without directly exchanging data, there are still potential privacy concerns. Furthermore, adversaries may be able to deduce some original data from the parameters of a model.

High-level FL privacy threats include inference attacks and communication bottlenecks. Secure multi-party computation, differential privacy, verifyNet, and adversarial training are effective techniques for preserving privacy in FL [88].

3.1.3 Convergence Guarantee. There have been extensive studies about the theoretical convergence properties of FL algorithms under different problem settings. Pioneer efforts along this line, such as [64, 83, 104], have analyzed the convergence speedup of FL algorithms and derived a desirable conclusion that linear speedup can be achieved for FedAvg, which is the most representative FL algorithm, with commonly adopted assumptions for analysis.

3.1.4 Communication Efficiency. To improve communication efficiency, one popular approach is to either reduce the number of communication rounds or to require fewer data to be transmitted per communication round [54]. Depending on the infrastructure of computing, by selecting an appropriate topology design, communication efficiency can also be optimized. Generally, the star topology network design ensures the least amount of communication with the central server since all devices are directly connected to it. In tree topologies, intermediate edge servers are usually involved, and devices can benefit from fast and efficient communication with edge devices at a low cost. For fully meshed topologies, communication usually takes place in a P2P or D2D manner, and direct communication between the devices is generally quite efficient. Furthermore, hybrid topologies are emerging, which combine the common topology with the strengths of each to produce a more dynamic system.

The aforementioned challenges in FL can be tackled in parallel. For instance, some personalized FL algorithms [62, 123], which share only partial model parameters, can tackle user heterogeneity while achieving high communication efficiency.

3.2 FL Characteristics Specific to Edge Computing

Unlike the typical configurations of FL, which follow a naive star topology, practical edge computing demonstrates unique characteristics in architectures and network configurations, which could deeply impact the design and implementation of the effective deployment of FL algorithms. We list the key features of edge computing frameworks below.

3.2.1 Heterogeneity, Energy Efficiency, and Task Offloading. A large portion of edge networks consists of user devices. These devices include highly embedded devices such as wearable glasses and watches, as well as powerful personal servers [116, 119, 150]. These devices are mostly still powered through batteries, making it necessary to consider *energy-efficient* protocol and algorithm design [43, 122]. The heterogeneous devices also introduce a huge variance of computational capacities, leading to a natural research direction of task offloading [13, 127].

The heterogeneity, energy efficiency, and task offloading play substantial roles in formulating the topologies of the edge networks [94]. To elaborate, energy consumption considerations prohibit a star-topology in a large-scale edge network because the central edge server would be overwhelmed due to its capability limit [79]. Offloading FL tasks from less capable edge devices to more

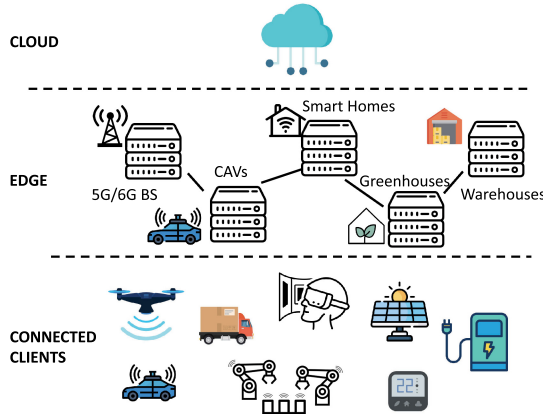


Fig. 4. Application scenarios of federated learning in mobile edge computing.

powerful edge devices is a viable and increasingly researched approach in FL and edge computing [13, 122, 127]. The offloading schemes are typically accompanied by their corresponding topology best practices [156, 157].

3.2.2 Hierarchy and Clustering. The nature of edge computing and 5G/6G communications has led to hierarchical networks where a base station covers the data transmission in small areas of wireless edge devices [19, 52]. The partial coverage results in multiple base stations deployed at the edge networks. The base stations can forward the data to the central server in a three-tier network hierarchy. When the number of base stations is large enough, there can be even more than three tiers for the data to move up along the hierarchy. The multi-fold hierarchy creates the prerequisites for configurable clustering and aggregation, making space for creativity over the hierarchical edge networks.

Hierarchical networks at the edge are often another product of the heterogeneous edge devices. Separations of capabilities have evolved into separations of hierarchies. Edge devices with lower capabilities can be dedicated to collecting sensing data and uploading it to their edge servers, whereas the edge server can be used for training local models and receiving updated global models. To minimize the exposure of the models to non-server parties, the edge servers can use different topology patterns from the central server to communicate with each other to maximize efficiency and privacy. In multi-tier edge networks, dynamic topologies can be applied based on internal and external factors for optimal learning performance.

3.2.3 Availability and Mobility. Compared to cloud data centers, edge servers have less redundancy and less reliability due to space, power, and budget. Mobile Edge devices, such as CAVs and **unmanned aerial vehicles (UAVs)**, have even lower availability because of their mobility. The moving edge device may enter and exit the boundaries of an edge network and switch between different clusters of an edge network, leading to interruptions of task processing and computations. Figure 4 shows some application scenarios of mobile edge computing in FL.

The mobile and volatile edge devices of CAVs and UAVs are pushing dynamic-format topology, where at any epoch of the system, there can be the addition or removal of edge devices and edge servers.

3.3 FL Challenges and Solutions in Edge Network Topologies

The unique characteristics of edge computing and edge networks are posing fundamental challenges to performing reliable and efficient federated learning and applying feasible distributed

learning systems at the edge. Many of those challenges can be resolved or mitigated by topology designs. In the following sections, we list some of the major challenges in FL and their corresponding solutions using a different network topology structure.

3.3.1 Scattered Data across Organizations. As its name describes, FL may require data from independent organizations to be federated. In this scenario, there are stricter data-sharing policies without directly sharing any data or the intermediate local models. For example, **federated transfer learning (FTL)** [74] can unite those organizations and leverage their data without violating privacy protection regulations. Compared with the vanilla FedAvg, FTL allows learning from the entire dataset, rather than only those samples with common features.

3.3.2 High Communication Costs. The original FL requires each device to directly communicate with the central server for upstream model aggregation and downstream model update. In the context of edge computing, direct communication to the central server is expensive for some edge devices and may cause high latency. The hierarchical edge computing topology can pool and aggregate the local updates from devices and hence reduce the communication costs to the cloud.

3.3.3 Privacy Concerns and Trust Issues. While federated learning keeps the storage of training data to the device, it still does not eliminate the risk of exposing sensitive information through repeated aggregated local model uploads to central servers. When a threat model considers the privacy concern in central aggregation servers, a network topology with decentralized model aggregation methods will help mitigate or eliminate the risk. The rationale behind this is that all relaying edge servers in the topology will aggregate part of the information. So one compromised central server will not be able to see all the fine-granularity model updates from all clients, and therefore largely reduces the differential information repeatedly exposed to the server.

3.3.4 Imbalanced Data Distribution. The nature of heterogeneous edge devices and networks in edge computing environments has led to significantly imbalanced data distribution and intensity based on the type of applications and devices. For example, an **augmented reality (AR)** application may generate a large burst of data over a short period when a user is actively using the application. In comparison, a temperature monitoring application may only generate a small amount of data for temperature records. However the data is produced constantly and periodically. By utilizing the tree network topology, methods like Astraea [24] add mediators between the FL server and the clients to resolve imbalanced data problems.

3.4 Categorization of Topology-Aware FL in Edge Computing

With the recent advancements in deep learning and increasing research interests in FL, a growing number of studies have expanded the horizon of FL applications. Numerous studies have reviewed existing FL areas [49, 61, 68, 88]. However, due to the broad applications and the nature of FL, there is no standard to summarize existing topology-aware FL studies systematically. Many existing FL studies focus on specific characteristics of FL and attempt to categorize it accordingly. Several of these methods are summarized in the following section.

3.4.1 Based on Data Partition Horizontal FL (HFL), vertical FL (VFL), and federated transfer learning (FTL). FL can be categorized into horizontal FL (HFL), vertical FL (VFL), and federated transfer learning (FTL) based on data partition in the feature and sample spaces [146]. Horizontal FL (HFL) represents a typical FL setting where the set of features of all participating clients are the same, making it easy for implementation and training. In most cases, studies treat horizontal FL as the default structure and may not even mention the term “horizontal”. For example,

the first implementation of FL by Google [83] is an example of HFL where the feature space of all participating devices is the same.

On the other hand, Vertical FL (VFL) [139] is catered specifically toward vertically partitioned data, where clients in VFL have different feature spaces. For example, hospitals and other health-care facilities may have data about the same patient but different types of health information. Fusing multiple types of information from the same set of samples or overlapping samples in different institutions belongs to the VFL setting.

FTL [74] was initially designed for scenarios where participants in FL have heterogeneous data in both feature space and sample space. In this setting, both HFL and VFL are unable to train efficiently in heterogeneous settings. FTL is considered the ideal solution at the time. FTL leverages the whole sample and feature space with transfer learning, where two neural networks serve as feature transformation functions that can project the source features of the two networks into a common feature subspace, allowing the knowledge to be transferred between the two parties.

3.4.2 Based on Model Update Protocols Synchronous, Asynchronous, and Semi-Synchronous FL. FL can be separated into synchronous, asynchronous, and semi-synchronous by communication protocols [121]. For synchronous FL [30, 75, 128], each learner performs a set round of local training. After every learner has finished their assigned training, they share their local models with the centralized server and then receive a new community model, and the process continues. Synchronous FL may result in the underutilization of a large number of learners and slower convergence, as others must wait for the slowest device to complete the training. With asynchronous FL [121, 141], there are no synchronization points. Instead, learners request community updates from the centralized server when their local training has been completed. As fast learners complete more rounds of training, they require more community updates, which would increase communication costs and lower the generalization of the global model. A semi-synchronous FL framework called FedRec [121] was proposed that allowed learners to continuously train on their local dataset up to a specific synchronization point where the current local models of all learners are mixed to form the community model.

3.4.3 Based on Data Distribution Non-IID and IID Data FL. One of the major statistical challenges surrounding FL in the early stage is when training data is non-IID [159]. The consistent performance of FL relies heavily on IID data distribution on the local clients. However, in most real-life cases, local data are likely non-IID, which significantly decreases the performance of existing FL techniques if not catered specifically to non-IID data. Therefore, existing FL studies can be categorized as FL with non-IID data or FL with IID data.

3.4.4 Based on Scale of Federation: Cross-Silo and Cross-Device FL. Based on the scale of the federation, FL studies can be divided into cross-silo and cross-device FL [46]. Cross-silo FL focus on coordinating a small amount of large data centers like hospitals or banks. On the other hand, Cross-device FL has relatively large amounts of devices and small amounts of data in each device. The key differences between the two are the number of participating parties and the amount of data stored in each participating party in FL.

3.4.5 Based on Global Model: Centralized and Decentralized FL. The most straightforward method for implementing and managing FL is to connect all participating devices through a central server. For centralized FL, the central server is either used to compute a global model or to coordinate local devices [93]. Having a central server, however, may contradict the aim of decentralization in FL. For fully decentralized FL, there is no overarching central server at the top, and devices are connected in a D2D or P2P manner.

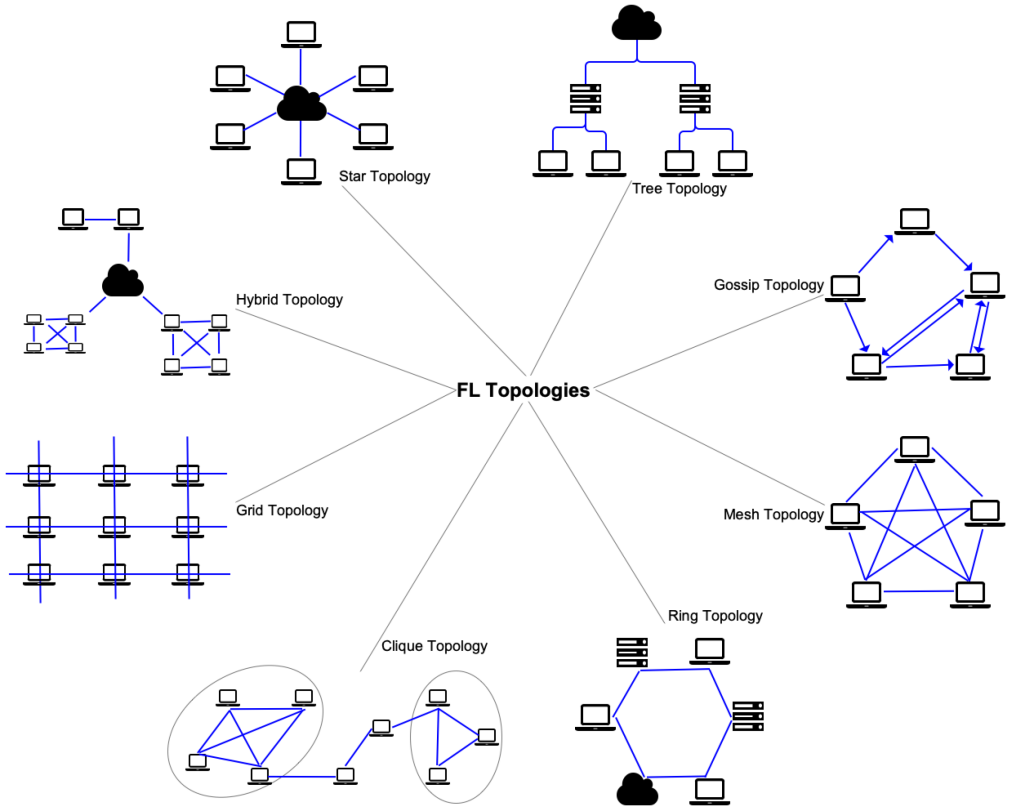


Fig. 5. Overview of FL topology types.

4 TYPES OF FL NETWORK TOPOLOGY

To inspire future research, our work summarizes state-of-the-art FL studies from the perspective of network topology, as opposed to existing FL reviews that focus on particular features, such as data partitioning, communication architecture, or communication protocols.

In the case of FL, the network topology represents how edge devices communicate with each other and eventually to a centralized server [154]. FL can benefit from the topologies of the networks to increase communication efficiency by performing partial and tiered model aggregations [9, 10], enhancing privacy by avoiding transmitting local models directly to a centralized server [162, 163], and improving scalability with horizontally replicable network structures [7, 11]. The major types of FL network topology essentially come down to either centralized (e.g., star topologies), decentralized (e.g., mesh topologies), or hybrid topologies, which consist of two or more traditional topology designs. Other less common network topologies, like ring topologies, will also be covered in this section. Figure 5 shows the overview of the mentioned FL topology.

4.1 Star Topology

The original use case of FL was to train machine learning algorithms across multiple devices in different locations. The key concept is to enable ML without centralizing or directly exchanging private user data. However, most FL implementations still require the presence of a centralized server. The most common network topology used in FL, including the original FL work [83], adopted centralized aggregation and distribution architecture, also known as “star topology”. As a result, the

Table 2. Highlighted Works - Star Topology

FL Type	Baselines and Benchmarks	Key Findings
Synchronous	FedAvg and Large Scale SGD with MNIST, CIFAR-10, CIFAR-100, and ILSVRC 2012	Computation and communication bandwidth were significantly decreased [30, 128]
	FedSGD, FedBCD-p, and FedBCD-s with MIMIC-III, MNIST, and NUS-WIDE	The models performed as well as the centralized model. Communication costs were significantly reduced [75]
	Noise-Free FL, Conventional RIS, Random STAR-RIS, Equal Power Allocation with MNIST, CIFAR-10 under IID and non-IID	STAR-RIS used both NOMA and AirFL framework to address the spectrum scarcity and heterogeneous services issues [93]
Asynchronous/ Semi-Synchronous	FedAvg and single-thread SGD with CIFAR-10 and WikiText-2	FedAsync was generally insensitive to hyperparameters, had fast convergence and staleness tolerance [141]
	FedAvg, FedAsync, and FedRec with Cifar-10 and Cifar-100	Faster generalization and learning convergence, better utilization of available resources and accuracy [121]
Personalized	eFD(Extended Federated Dropout) and Federated Dropout(FD) using CIFAR10, FEMNIST, and Shakespeare	Able to extract submodels of varying FLOPs and sizes without the retraining; flexibility across different environment setups [37]
	pFedMe, Ditto, FedAlt, and FedSim with StackOverflow, EMNIST, GLDV2, and LibriSpeech	Proposed partial model personalization can obtain most benefit of full model personalization; provided convergence guarantee [98]
	FedAvg, pFedMe, Ditto, FedEM, FedRep, FedMask, and HeteroFL with EMNIST, FEMNIST, CIFAR10, and CIFAR100	Significantly improves performance; thorough theoretical analysis; extensive experiments are conducted show superior effectiveness, efficiency, and robustness [12]

graph of the server-client architecture resembles a star. Numerous FL research and algorithms are based on the assumption of a star topology [30, 93, 128]. While being the most straightforward approach, a star network topology suffers from issues like high communication costs, privacy leak concerns to the central server, and security concerns [26]. Some studies posed solutions to address these issues [93]. However, the star-topology-based solutions are not always the optimal network topology design for all FL systems. It is worth questioning if the star architecture is the network topology that best fits all scenarios.

There are a substantial number of studies in FL using the default star network topology [46, 63, 68, 70, 146, 150]. Most of those studies do not focus on the aspect of network topology or edge computing. In this section, we select various FL works that focus on optimizing the topology, communication cost, and edge computing while still using the traditional star topology structure. In Table 2, we highlight some of the works using star topology.

A distributed learning method called splitNN was proposed [30, 128] to facilitate collaborations of health entities without sharing the raw health data. In a star topology, all subsequent nodes are connected to the master node. Data does not have to be shared directly with the master node. By using a single supercomputing resource, a star topology network can provide training with access to a significantly larger amount of data from multiple sources. Alice(s) represent data entities in the deep neural network, and Bob represents one supercomputing resource that corresponds to the role of nodes and a central server. While all the single data entities (Alices) are connected to the supercomputing resource (Bob), no raw data are shared between each other. Techniques will include encoding data into a different space and transmitting it to train a deep neural network. Experimental results were obtained on the MNIST, CIFAR-10, and ILSVRC (ImageNet) 2012 datasets and showed similar performance to other neural networks trained on a single machine. As compared with classic single-agent deep learning models, this technique significantly reduces client-side computational costs. Although federated learning was available at the time, the authors argued

that there had been no proper non-vanilla settings with vertically partitioned data and without labeling, with distributed semi-supervised learning and distributed multi-task learning.

An algorithm named **Federated Stochastic Block Coordinate Descent (FedBCD)** [75] was proposed boasting multiple local updates before communications to the central server. Through theoretical analysis, the authors found that the algorithm needed $O(\sqrt{T})$ iterations for T iterations and achieved $O(1/\sqrt{T})$ accuracy.

Ni et al. [93] proposed a new FL framework called STAR-RIS which integrates **nonorthogonal multiple access (NOMA)** and **over-the-air federated learning (AirFL)**. STAR-RIS used NOMA and AirFL frameworks to address the spectrum scarcity and heterogeneous services issues in FL. This work follows the classical star topology where all client needs to update in a synchronized fashion and connect to the server. The proposed STAR-RIS used a novel approach that utilizes simultaneous transmitting and reflecting reconfigurable intelligent surface that boosts performance compared with other methods. STAR-RIS addressed issues specific to the integration of communication and learning technologies for the 6G network. STAR-RIS provided a closed-form expression for the convergence upper bound, which gives a strong theoretical guarantee.

4.1.1 Asynchronous FL Topologies. Stripelis and Ambite [121] identified the issue that in heterogeneous environments, classic FL approaches exhibited poor performance. Synchronous FL protocols were communication efficient but had slow learning convergence, while asynchronous FL protocols had faster convergence but higher communication costs. For synchronous FL, the original FedAvg algorithm serves as a great example: after each participant device trains for a fixed number of epochs, the system will wait until all the devices complete their training and then compute the community models. The approach is in no way efficient, but it limits communication to a fixed amount because all devices will have the same number of communication rounds. In particular, in the case of fast and slow workers, the fast devices will have a long idle time to wait for the slow devices. For asynchronous FL, FedAsync [141] provides a thorough analysis of the subject. asynchronous FL is the complete opposite of synchronous FL. As the previous protocol minimizes communication costs, asynchronous protocols seek to utilize all participant devices to their fullest capability, which means once a device finishes assigned training, it can request a community update and continue training. However, this approach significantly increases network communication costs for fast devices. Semi-synchronous [121] FL seeks to combine the benefits of both protocols by setting up a synchronization point for all devices, allowing the fast devices to complete more rounds of training and prevent excessive communication along the way.

4.1.2 Personalized Star Topology. There has been great attention to personalized FL in recent studies, mainly to increase the fairness and robustness of FL [124]. Most personalized FL [12, 37, 62, 124] follows the traditional star topology. One interesting aspect of personalized FL is that personalized local clients may require fewer models to be transmitted over the network, one approach is to partially upload and download the global models from the server [98]. Another approach is to have a dynamically adapting model size based on the heterogeneous data distributions or resource constraints [12, 37]. From the topology perspective, personalized FL brings some unique opportunities for further optimizing communication with various-sized local models.

Pillutla et al. [98] explored the idea of training partially personalized models, each local model has some shared and personal parameters. The authors experimented with both the simultaneous update and alternating update approaches. In addition, there is another personalized FL known as pFedme [123], which employs Moreau envelopes as a way of regularizing loss functions. pFedMe follows the same structure as the conventional FedAvg algorithm with an additional parameter used for the global model update. Specifically, each client must solve to obtain their personalized mode, which is used for local updates. The server uniformly samples a subset of clients and the

Table 3. Features and Benefits of Tree Topology

Features	Benefits
Clustered clients	Adaptive strategies of in-cluster communications based on cluster's condition.
Configurable cluster	Better scalability compared to star topology.
Configurable number of layers	Varying policies for client-edge and edge-cloud, and inter-layer aggregations.

local model is sent to the server. Horvath et al. [37] proposed Fjord which dynamically adapts the model size by with Ordered Dropout. By using this importance-based pruning approach, Fjord can create nested submodels from a main model and enable partial training only on the submodels. Fjord shows strong scalability and adaptability compared with baseline methods. Chen et al. [12] take a further step on personalized FL by optimizing both clients' local data distribution and hardware resources using adaptive gated weights. The proposed pFedGate [12] can generate personalized sparse models while also considering the resource limitation of the local device. Combining both model compression and personalization approaches, pFedGate achieves superior global and individual accuracy and efficiency compared to existing methods.

4.1.3 Cohorts and Secure Aggregation. Charles et al. [11] studied how the number of clients sampled at each round affected the learning model. Challenges were encountered while using large cohorts in FL. Particularly, the data heterogeneity caused the misalignment between the server model x and the client's loss f_k . With a threshold for "catastrophic training failure" defined, the authors revealed that the failure rate increased from 0% to 80% when the cohort size expanded from 10 to 800. While the star topology remained the same, improved methods were proposed, including dynamic cohort sizes [120], scaling the learning rate [28, 56].

Secure aggregation protocols with poly-logarithmic communication and computation complexity were proposed in [3] and [16] requiring three rounds of interaction between the server and clients. In [6], the star topology of the communication network was replaced with a random subset of clients and secret sharing was only used for a subset of clients instead of all client pairs. Shamir's t-out-of-n Secret Sharing technique prevents the splitted subgroups from divulging any information about the original. In [16], the proposed **secure aggregation (CCESA) algorithm** provided data privacy using substantially reduced communication and computational resources compared to other secured solutions. The key idea was to design the topology of secret-sharing nodes as a sparse random graph instead of the complete graph [6]. The required resource of CCESA is reduced by a factor of at least $O(\sqrt{n/\log n})$ compared to [6].

4.2 Tree Topology

There can be additional layers between the central server and edge devices. For instance, edge servers that connect edge devices and the central server can formulate one or multiple layers, making a tree-like topology with the highest level of the tree being the central server and the lowest level being edge devices. Tree topologies must contain at least three levels. Otherwise, they are considered star topologies. Compared to traditional FL, tree topology helps overcome performance bottlenecks and single points of failure. We list the features and benefits of tree topology in Table 3. In this section, we discuss applications and motivations for adopting tree topology. A review of state-of-the-art optimization frameworks and algorithms is presented. We show some visualization of tree topology structures and their benefits for FL in Figure 6 and Figure 7. In the end, we cover some grouping strategies and privacy enhancement schemes.

There are two major categories of FL studies in tree topology: hierarchical and dynamic. Hierarchical represents the classic two-tier hierarchy in topology design, while dynamic continues to

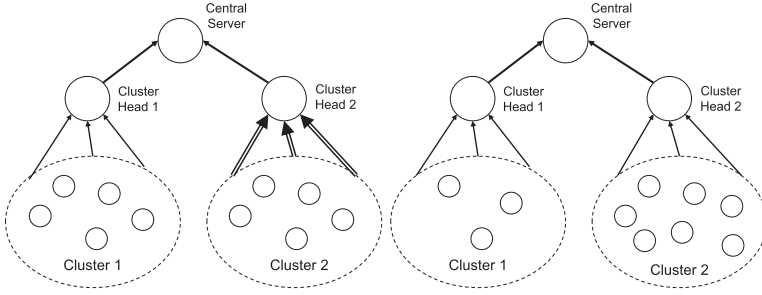


Fig. 6. **Right:** FL with tree topology enables varying communication costs in different clusters depending on their energy profiles. **Left:** The cluster size can be different for tree topology based on data distribution and other parameters of each cluster.

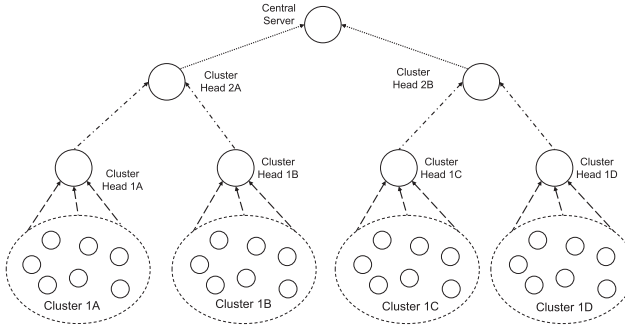


Fig. 7. There can be arbitrary layers of clients in the tree topology FL system.

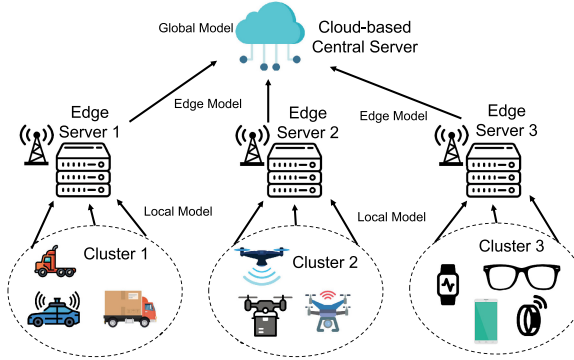


Fig. 8. Hierarchical FL following tree topologies. Typically, the FL network has a tree structure with at least three tiers: the cloud tier, the edge tier, and the device tier.

follow the overall structure of the tree topology with some modifications. In the following sections, we organize the works that use the tree topology based on their topics. We show a classic example of Hierarchical FL in Figure 8. We list the features and benefits of tree topology in Table 3. We highlight some of the works using tree topology in Table 4 and Table 5.

4.2.1 Typical Tree Topology FL. Zhou et al. [163] proposed a typical use of end-edge-cloud federated learning framework in 6G. The authors integrated a convolutional neural network-based

Table 4. Highlighted Works - Tree Topology - Hierarchical

FL Type	Baselines and Benchmarks	Key Findings	Performance
Hierarchical	Hierarchical FL using CNN and mini-batch SGD with MNIST and CIFAR-10 under non-IID setting	Vanilla hierarchical FL, ignores heterogeneous distribution	Reduced communication, training time, and energy cost with the cloud. Also achieved efficient client-edge communication [71]
	Resource allocation methods and FedAvg with MNIST and FEMNIST	Multiple edge servers can be accessed by the device. Optimize device computation capacity and edge bandwidth allocation	Better global cost-saving, training performance, test and training accuracy, and lower training loss than FedAvg [78]
	Binary tree and static saturated structure, and FSVRG and SGD algorithm with MNIST	Using the layer-by-layer approach, more edge nodes can be included in the model aggregation	Scalability (time cost increases logarithmically rather than linearly in traditional FL), reduced bandwidth usage and time-consuming [8]
	Uniform, gradient-aware, and energy-aware scheduling with MNIST	Optimize scheduling and resource allocation by striking a balance between 3 scheduling schemes	Outperformed the baselines if λ is chosen properly. Otherwise slightly better or worse performance [140]
	FedAvg plus SGD using CNN with MNIST	Both the central server and the edge servers are responsible for global aggregation	Reduced global communication cost, model training time and energy consumption [147]
	RF, CNN, and RegionNet with BelgiumTSC	Classic hierarchical FL in 5G and 6G settings for object detection	Faster convergence and better learning accuracy for 6G supported IoV applications [163]
	FedAvg with imbalanced EMNIST and CINIC-10, CIFAR-10	Relieved global and local imbalance of training data; recover accuracy	Significantly reduced communication cost and achieved better accuracy on imbalanced data [24]
	FedAvg with MNIST and FEMNIST under IID and non-IID settings	A clustering step was introduced to determine client similarity and form subsets of similar clients	Fewer communication rounds especially for some non-IID settings. Allowed more clients to reach target accuracy [7]

approach specially perform hierarchical and heterogeneous model selection and aggregation using individual vehicles and RSUs at the edge and cloud level. Evaluation results showed an overall better outperform in learning accuracy, precision, recall, and F1 score compared to other state-of-the-art methods in 6G network settings.

Yuan et al. [151] designed a LAN-based hierarchical federated learning platform to solve the communication bottleneck. The authors regarded that existing FL protocols have a critical communication bottleneck in a federated network coupled with privacy concerns, usually powered by a **wide-area network (WAN)**. Such a WAN-driven FL design led to significantly higher costs and much slower model convergence. An efficient FL protocol was proposed to create groups of LAN domains in P2P mode without an intermediate edge server which involved a hierarchical aggregation mechanism using **Local Area Network (LAN)**, as it had abundant bandwidth and almost negligible cost compared to WAN.

The benefits of training data aggregation at the edge in HFL were acknowledged by Deng et al. [20]. When comparing HFL and cloud-based FL, κ_e and κ_c were defined as the aggregation frequency at the edge and the cloud, respectively. It was concluded from their research that in the HFL framework, with fixed κ_e and κ_c , *uniform distribution* of the training data at the edge significantly enhanced FL performance and reduced the rounds of communications. The original problem was first divided into two sub-problems to minimize the per-round communication cost and mean

Table 5. Highlighted Works - Tree Topology - Dynamic

FL Type	Baselines and Benchmarks	Key Findings	Performance
Dynamic	FedAvg using Random and heuristic sampling with MNIST and F-MNIST	Able to offload data from non-selected devices to selected devices during training	Significant improvements in datapoints processed, training speed, and model accuracy [132]
	FedAvg using F-Fix and F-Opt with CNN on MNIST	Flexible system topology that optimizes computing speed and transmission power	Accelerated the federated learning process, and achieved a higher energy efficiency [42]
	WAN-FL using CNN with FEMNIST and CelebA under non-IID settings	Dynamic device selection based on the network capacity of LAN domains. Relied heavily on manual parameter tuning	Accelerate training process, saved WAN traffic, and reduced monetary cost while preserving model accuracy [151]
	FedAvg, TiFL, FedAsync with FMNIST, CIFAR-10, Sentiment140	Models were updated synchronously with clients of the same tier and asynchronously with the global model across tiers	Faster convergence towards the optimal solution, improved prediction performance, and reduced communication cost [10]
	Cloud-based FL (C-FL), Cost only CPLEX (CC), Data only greedy (DG) with MNIST and CIFAR-10	As opposed to an edge server, groups of distributed nodes are used for edge aggregation	Improved FL performance at a very low communication cost, provided a good balance between learning performance and communication costs [20]
	Traditional FL (TFL) low and high power mode with MNIST under IID and non-IID settings	Based on the status of their local resources, clients are assigned to different subnetworks of the global model	Outperformed TFL in both low and high power modes, especially in low power. Reliable in dynamic wireless communication environments [149]

Kullback–Leibler divergence (KLD) of edge aggregator data. Then two lightweight algorithms were developed, adopting a heuristic method formulating a topology encouraging a uniform distribution of training data.

Briggs et al. pointed out in [7] that in reality, most data was distributed in a non-IID fashion. These fashions included feature distribution skew, label distribution skew, and concept shifts. In such cases, most FL methods suffered accuracy loss. They introduced a hierarchical clustering step (FL+HC) to separate clusters of clients by the similarity of their local updates to the global joint model. Then multiple models targeted toward groups of clients were preferred. The empirical study showed that FL+HC allowed the training to converge in fewer communication rounds with higher accuracy.

4.2.2 Optimization: Trade-off among Energy Cost, Communication Delay, Model Accuracy, Data Privacy. Liu et al. [71] proposed a client-edge-cloud hierarchical learning system that reduced communication with the cloud by trading off between the client-edge and edge-end communication costs. This is achieved by leveraging the edge server’s ability to exchange local updates with clients constantly. There are two types of data collection rounds: one is from the client to the edge servers, and the other is from the edge servers to the cloud. The proposed FL algorithm **Hierarchical Federated Averaging (HierFAVG)** extends from the classic FAVG algorithm. Under the HierFAVG architecture, after the local clients finish k_1 rounds of training, each corresponding edge server aggregates its client’s model for k_2 rounds of aggregation, and the cloud server then aggregates all the edge servers’ models. Compared to the traditional systems following the star topology, this tree topology-based architecture greatly reduced the total communication rounds with the cloud server. Standard MNIST and CIFAR-10 datasets were used for the experiment. Additional

two non-IID cases for MNIST were also considered. Experiments showed promising results on reduced communication frequency and energy consumption. When the overall communication ($k_1 k_2$) is fixed, fewer rounds of local updates (k_1) and more communication rounds with the edge will result in faster training which effectively reduces the number of computation tasks on the local clients. For the case of IID data on the edges, fewer communication rounds with the cloud server will not result in a decrease in performance as well. On energy consumption, with moderately increased communication with clients and edge, the energy consumption decreases. However, excessive communication between edge servers and clients will result in extra energy consumption. Therefore, a balance of overall communication ($k_1 k_2$) is needed to minimize energy consumption.

Luo et al. [78] also introduced a **Hierarchical Federated Edge Learning (HFEL)** framework to jointly minimize energy consumption and delay. The authors formulated a joint computation and communication resource allocation problem for global cost optimization, which considered minimizing system-wide energy and delay within one global iteration, denoted by the following equation:

$$E = \sum_{i \in \kappa} (E_i^{cloud} + E_{S_i}^{edge}), \quad (1)$$

$$T = \max_{i \in \kappa} \{T_i^{cloud} + T_{S_i}^{edge}\}, \quad (2)$$

where E represented the total energy consumed by each edge server $i \in \kappa$ aggregating data and each sets of devices S_i for edge server i uploading models. T was defined as the total delay with those introduced by the edge servers to the cloud, denoted by T_i^{cloud} , and by the sets of devices uploading models, denoted by $T_{S_i}^{edge}$. The optimization was to jointly minimize E and T with varying weights. A resource scheduling algorithm was developed based on the model, which relieved the core network transmission overhead and enabled great potential in low-latency and energy-efficient FL.

Cao et al. proposed a federated learning system [8] with an aggregation method using the topology of the edge nodes to progress model aggregation layer by layer, specifically allowing child nodes on the lower levels to complete training first and then upload results to the higher node. Compared to the traditional FL architecture, where all the end devices connect to the same server, the proposed layered and step-wise approach ensures that only one gradient data is transmitted in a link at most. The simulation result shows better scalability where the time cost increases logarithmically rather than linearly in traditional FL systems.

Another joint optimization strategy that investigated the trade-off between computation cost and accuracy was presented by Wen et al. [140] for **hierarchical federated edge learning (HFEEL)**, where an optimization approach was developed to minimize the weighted sum of energy consumption and gradient divergence. The innovative contributions included three phases: local gradient computing, weighted gradient uploading, and model updating.

Ye et al. [147] proposed EdgeFed featuring the trade-off between privacy and computation efficiency. In the EdgeFed scheme, split training was applied to merge local training data into batches before being transmitted to the edge servers. Local updates from mobile devices were partially offloaded to edge servers where more computational tasks are assigned, reducing the computational overhead to mobile devices, which can focus on the training of *low layers*. In the EdgeFed algorithms, each iteration included multiple split training between K edge devices and corresponding edge servers and a global aggregation between m edge servers and the central server. The edge device k performed calculations with local data on low layers of the multi-layer neural network model. After receiving the outputs of low layers from all edge devices, the edge server m aggregated all data received into a larger matrix x_{pool}^m , which was then taken as the input of the remaining

layers:

$$x_{pool}^m \leftarrow [x_{conv}^1, x_{conv}^2, \dots, x_{conv}^k, \dots, x_{conv}^K] \quad (3)$$

As the updates to the central server were narrowed down between edge servers and the central server, with more computational power on the edge server compared to edge devices, the overall communication costs were reduced. However, the transferred data processed by the low layer of the model may be a threat to privacy because the edge server may be able to restore the original data.

4.2.3 Dynamic Topology. Mhaisen et al. [84] proposed that the tree topology can be dynamic with a dense edge network. Edge devices may pair with different edge servers in different rounds of data aggregation. When participants change either due to the client selection strategy [106, 132] or participants entering or exiting the network [55], the topology will subsequently change. The authors argued that **user equipment (UE)** had access to more than one edge server in dense networks and increased the mobility of UE. Choosing the best edge server resulted in their proposed UE-edge assignment solutions. The user assignment problem was formalized in HFL based on the analysis of learning parameters with non-IID data.

Kourtellis et al. [55] explored the possibility of collaborative modeling across different 3rd-party applications and presented **federated learning as a service (FLaaS)**, a system allowing 3rd-party applications to create models collaboratively. A proof-of-concept implementation was developed on a mobile phone setting, demonstrating 100 devices working collaboratively for image object detection. FedPAQ was proposed in [106] as a communication-efficient federated learning method with **periodic averaging and quantization**. FedPAQ's first key feature was to run local training before synchronizing with the parameter server. The second feature of FedPAQ was to capture the constraint on the availability of active edge nodes by allowing partial node participation, leading to better scalability and a smaller communication load. The third feature of FedPAQ was that only a fraction of device participants sent a quantized version of their local information to the server during each round of communication, significantly reducing the communication overhead.

The device sampling in Heterogeneous FL was studied in [132]. The authors noticed that there may be significant overlaps in the local data distribution of devices. Then a joint optimization was developed with device sampling aiming at selecting the best combination of sample nodes and data offloading configurations to maximize FL training accuracy with network and device capability constraints.

Huang et al. [42] proposed a novel **topology-optimized federated edge learning (TOFEL)** scheme where any devices in the system received and aggregated their own gradients and then passed them to other devices or edge servers for further aggregation. The system acted as a hierarchical FL topology with adjustable gradient uploading and aggregation topology. The authors formulated a joint topology and computing speed optimization as a **mixed-integer nonlinear program (MINLP)** problem aims at minimizing energy consumption and latency. A penalty-based **successive convex approximation (SCA)** method was developed to transform the MINLP into an equivalent continuous optimization problem which demonstrates that the proposed TOFEL scheme speed up the federated learning process while consuming less energy.

Duan et al. [24] focused on the imbalanced data distribution in mobile systems, which led to model biases. The authors built a self-balancing FL framework called Astraea to alleviate the imbalances with a mediator to reschedule the training of clients. The methods included Z-score-based data augmentation and mediator-based multi-client rescheduling. The Astraea framework consisted of three parts: FL server, mediator, and clients.

4.2.4 Grouping Strategy and Privacy Enhancement. He et al. [34] proposed a grouping mechanism called Auto-Group, which automatically generated grouped users using an optimized Genetic Algorithm without the need to specify the number of groups. The Genetic Algorithm balanced the data distribution of each group to be as close as the global distribution.

FedAT was a method with **asynchronous tiers** under non-IID data proposed by Chai et al. [10]. FedAT had the topology organized in tiers based on the response latencies of edge devices. For intra-tier training, the synchronous method was used as the latencies are similar. For cross-tier training, the asynchronous method was used. The clients were split into tiers based on their training speed with the help of the tier-based module, allowing faster clients to complete more local training while using server-side optimization to avoid bias. By bridging the synchronous and asynchronous training through tiering, FedAT minimized the straggler effect with improved convergence speed and test accuracy. FedAT used a straggler-aware, weighted aggregation heuristic to steer and balance the training for further accuracy improvement. FedAT compressed the up-link and downlink communications using an efficient, polyline-encoding-based compression algorithm, therefore minimizing the communication cost. Results showed that FedAT improved the prediction performance by up to 21.09%, and reduced the communication cost by up to 8.5 times compared to the state-of-the-art FL methods.

With the two typical FL scenarios in MEC, i.e., virtual keyboard and end-to-end autonomous driving, Yu and Li proposed a neural-structure-aware resource management approach [149] for FL. The mobile clients were assigned to different subnetworks of the global model based on the status of local resources.

Wainakh et al. [130] discussed the implications of the hierarchical architecture of edge computing for privacy protection. The topology and algorithm enabled by hierarchical FL (HFL) may help enhance privacy compared to the original FL. These enhancements included flexible placement of defense and verification methods within the hierarchy and the possibility of employing trust between users to mitigate several threats. The methods linked to HFL were illustrated, such as sampling users, training algorithms, model broadcasting, and model updates aggregation. Group-based user update verification could also be introduced with HFL. Flexible applications of defense methods were available in HFL because of the hierarchical nature of the network topology.

4.3 Decentralized/Mesh Topology

Decentralized/Mesh topology is a network topology where all end devices are inter-connected to each other in a local network [15, 58, 73, 118, 142, 152]. In recent studies, mesh topologies are commonly used in FL systems. Decentralized approaches like **peer-to-peer (P2P)** or **device-to-device (D2D)** FL fall under the Mesh Topology. Many existing FL systems still rely on a centralized/cloud server for model aggregation [54]. The decentralized approach is sometimes regarded as a poor alternative to the centralized method when a centralized server is not feasible. This section covers three major FL systems using fully decentralized approaches. In Table 6, we highlighted some works that utilized the decentralized topology.

The performance of decentralized FL algorithms was discussed in [67]. Theoretical analysis on a D-PDGD algorithm was conducted to prove the possibility of a decentralized FL algorithm outperforming centralized FL algorithms. With comparable computational complexity, the decentralized FL algorithm required much less communication cost on the busiest node. D-PSGD could be one order of magnitude faster than its well-optimized centralized counterparts.

Lu et al. [77] developed a vehicular FL scheme based on a sub-gossip update mechanism along with a secure architecture for **vehicular cyber-physical systems (VCPS)**. The P2P vehicular FL scheme used random sub-gossip updating without a curator, which enhanced security and efficiency. The aggregation process was done in each vehicle asynchronously. The data retrieval

Table 6. Highlighted Works - Decentralized Topology

FL Type	Baselines and Benchmarks	Key Findings
Decentralized Mesh	Using 20 Newsgroups dataset integrating GBDT	Obtained high utility and accuracy, effective data leakage detection, near-real-time performance data leakage defending [77]
	FedAvg and FedGMTL using AGE and GAT with MoleculeNet	Train GNNs in serverless scenarios, outperformed Star FL even if clients can only communicate with few neighbors [32]
	PENS, Random, Local, FixTopology, Oracle, IFCA, FedAvg with MNIST, FMNIST, and CIFAR10	CNI was effective to match neighbors with similar objectives; directional communications helped to converge faster; robust in non-IID settings [66]
	FedAvg using ResNet-20 model with CIFAR-10 under IID and non-IID settings	Provided an unbiased estimate of the model update to PS through relaying; optimized consensus weights of clients to improve convergence; compatible in different topologies [148]
Decentralized Wireless	FedAvg, CDSGD, D-PSGD using CNN with MNIST, FMNIST, CIFAR-10 under IID and non-IID settings	Outperformed in accuracy, less sensitive to the topology sparsity; similar performance for each user; viable on IID and non-IID data under time-invariant topology [15]
	DSGD, TDMA-based, local SGD(no communication) with FMNIST	Over-the-air computing can only outperform conventional star topology implementations of DSGD [142]
	DOL and COL with SUSY and Room Occupancy dataset	Worked better than DOL in row stochastic confusion matrix, usually outperformed COL in running time [33]
	FedAvg and gossip approach without segmentation with CIFAR-10	Required the least training time to achieve given accuracy, more scalable, synchronization time significantly reduced [40]
	Gossip and Combo with FEMNIST and Synthetic data	Maximize bandwidth utilization by segmented gossip aggregation over the network; speed up training; maintain convergence [45]
	DFL and C-SGD with MNIST, CIFAR-10	Showed linear convergence behavior for convex objective, strong convergence guarantees for both DFL and C-DFL [73]
	FLS with MALC dataset using QuickNAT architecture	Enabled more robust training; similar performance with centralized approaches; generic method and transferable [108]
	FedAvg with MNIST using CNN and LSTM	Improved convergence performance of FL especially when the model was complex and network traffic was high [99]
	Gossip and GossipPGA using LEAF with FEMNIST and Synthetic data	Reduced training time and maintained good convergence, whereas partial exchange significantly reduced latency [44]

information was registered on nearby RSUs as a **distributed hash table (DHT)**. All related vehicles were searched for the DHT before FL started.

Gossip learning [35, 95] was compared with FL in [36] as an alternative, where training data also remained at the edge devices, but there was no central server for aggregation. Gossip learning can be seen as a variation of the mesh topology. Nodes exchanged and aggregated models directly. No centralized servers meant no single-point failure and led to better scalability and robustness. The performance of gossip learning was generally comparable with FL and even better in some scenarios. The experiment was conducted using PEERSIM [87].

SpreadGNN was proposed in [32] as a novel multi-task federated training framework able to operate with partial labels of client data for graph neural networks in a fully decentralized manner. The serverless multitask learning optimization problem was formulated, and **Decentralized Periodic Averaging SGD (DPA-SGD)** was introduced to solve the problem. The result shows that it is viable to train graph neural networks federated learning in a fully decentralized setting.

Li et al. [66] leveraged P2P communications between FL clients without a central server and proposed an algorithm formulating a decentralized, effective communication topology in a decentralized manner without assuming the number of clusters. To design the algorithm, two novel metrics were created for measuring client similarity. Another two-stage algorithm directed the clients to match same-cluster neighbors and to discover more neighbors with similar objectives. Theoretical analysis was included showing the effectiveness of the work compared to other P2P FL methods.

A semi-decentralized topology was introduced by Yemini et al. [148], where a client was able to relay the update from its neighboring clients. A weighted update with both the client's own data and its neighboring clients' data was transmitted to the **parameter server (PS)**. The goal was to optimize averaging weights to reduce the variance of the global update at the PS, as well as minimize the bias in the global model, eventually reducing the convergence time.

4.3.1 Decentralized Topology in Wireless Networks. The mesh or decentralized FL topology has been explored in wireless networks [27, 118, 142, 145], as the wireless coverage of P2P or D2D devices overlaps with each other, plus no centralized server has been provided.

Trust was treated as a metric of FL in [27]. The trust was quantified upon the relationship among network entities according to their communication history. Positive contributions to the model were interpreted as an increment of trust, and vice versa.

Shi et al. proposed over-the-air FL [118] over wireless networks, where **over-the-air computation (AirComp)** [145] was adopted to facilitate the local model consensus in a D2D communication manner.

Chen et al. [15] considered the deficiency of high divergence and model average necessity in previous decentralized FL implementations like CDSGD and D-PSGD. They devised a **decentralized FL implementation** called **DACFL** [15] which adapts more to non-ideal network topology. DACFL allows individual users to train their own model with their own training data while exchanging the intermediate models with neighbors using **FODAC (first-order dynamic average consensus)** to negate potential over-fitting problems discretely without a central server during training.

Xing et al. considered a network of wireless devices sharing a common fading wireless channel for deploying FL [142]. Each device held a generally distinct training set, and communication typically took place in a D2D manner. In the ideal case, where all devices within their communication range could communicate simultaneously and noiselessly, a standard protocol guaranteed the convergence to an optimal solution of the global empirical risk minimization problem under convexity and connectivity assumptions was called the **Decentralized Stochastic Gradient Descent (DSGD)**. DSGD integrated local SGD steps with *periodic consensus averages* that required communication between neighboring devices. Wireless protocols were proposed for implementing DSGD by accounting for the presence of path loss, fading, blockages, and mutual interference.

He et al. explored the use cases of FL in social networks where centralized FL was not applicable [33]. **Online Push-Sum (OPS)** method was proposed to leverage trusted users for aggregations. OPS offered an effective tool to cooperatively train machine learning models in applications where the willingness to share is single-sided.

Lalitha et al. considered the problem of training models in fully decentralized networks. They proposed a distributed learning algorithm [58] where users aggregated information from their on-hop neighbors to learn a model that best fits their observations to the entire network with small probabilities of error.

Savazzi et al. proposed a fully distributed, serverless FL approach to massively dense and fully decentralized networks [113]. Devices are independently trained based on local datasets received

from neighbors. Then devices forwarded the model updates to their one-hop neighbors for a new consensus step, extending the method of gossip learning. Both model updates and gradients were iteratively exchanged to improve convergence and minimize the rounds of communications.

Combo, a decentralized federated learning system based segmented gossip approach was presented in [40] to split the FL model into segmentations. A worker updated its local segmentation with k other workers, where k was much smaller than the total number of workers. Each worker stochastically selected a few other workers for each training iteration to transfer model segmentation. Replication of models was also introduced to ensure that workers had enough segmentation for training purposes.

Jiang et al. proposed **Bandwidth Aware Combo (BACombo)** [45] with a segmented gossip aggregation mechanism that makes full use of node-to-node bandwidth to speed up the communication time. Besides, a bandwidth-aware worker selection model further reduces the transmission delay by greedily choosing the bandwidth-sufficient worker. The convergence guarantees were provided for BACombo. The experimental results on various datasets demonstrated that the training time was reduced by up to 18 times that of baselines without accuracy degradation.

The work in [73] focused on balancing between communication-efficiency and convergence performance of **Decentralized federated learning (DFL)**. The proposed framework performed both multiple local updates and multiple inter-node communications periodically, unifying traditional decentralized SGD methods. Strong convergence guarantees were presented for the proposed DFL algorithm without the assumption of a convex objective function. The balance of communication and computation rounds was essential to optimize decentralized federated learning under constrained communication and computation resources. To further improve the communication efficiency of FL, compressed communication was applied to DFL, which exhibited linear convergence for strongly convex objectives.

BrainTorrent [108] was proposed to perform FL for medical centers without the use of a central server to protect patient privacy. As the central server required trust from all clients, which was not feasible for multiple medical organizations, BrainTorrent presented a dynamic peer-to-peer environment. All medical centers directly interact with each other, acting like a P2P network topology. Each client maintains its model version of the model and the last versions of models it used during merging. By sending a ping request, a client receives responses from other clients with their latest model versions and subsets of the models. The client then merged the models received by weighted averaging to generate a model.

FedAir [99] explored enabling FL over wireless multiple-hop networks, including the widely deployed wireless community mesh networks. Wireless multi-hop FL system consists of a central server as an aggregator with multi-hop link wireless to edge servers as workers. According to the authors, multi-hop FL faced several challenges, including slow convergence rate, prolonged per-round training time and potential divergence of synchronous FL, and difficulties in model-based optimization for multiple hops.

Jiang and Hu proposed **gradient partial level decentralized federated learning (FedPGA)** [44] aiming to improve on traditional star topology FL's high training latency problem in real-world scenarios. The authors used a partial gradient exchange mechanism to maximize the bandwidth to improve communication time, and an adaptive model updating method to adaptively increase the step size. The experimental results showed up to $14 \times$ faster training time compared to baselines without compromising accuracy.

4.3.2 Routing in Decentralized Topology. A topology design problem for cross-silo FL was analyzed in [82] due to traditional FL topology designs being inefficient in the cross-silo settings. They proposed algorithms that find the optimal topology using the theory of max-plus linear systems.

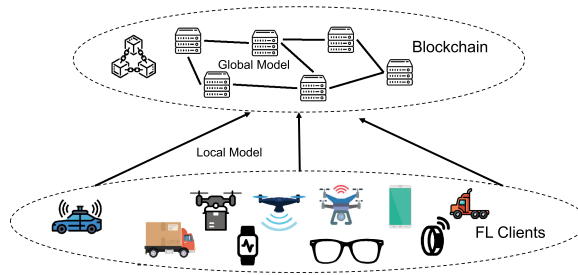


Fig. 9. FL with blockchain as distributed ledgers to increase the availability of the central server.

By minimizing the duration of communication rounds or maximizing the largest throughput, they were able to find the most optimal topology design that significantly shortens training time.

Sacco et al. proposed Blaster [110], a federated architecture for routing packets within a distributed edge network, to improve the application's performance and allow scalability of data-intensive applications. A path selection model was proposed using **Long Short Term Memory (LSTM)** to predict the optimal route. Initial results were shown with a prototype deployed over the GENI testbed. This approach showed that communications between SDN controllers could be optimized to preserve bandwidth for the data traffic.

In the Cross-device FL scenario, Ruan et al. studied flexible device participation in FL [109]. The authors assumed that it was difficult to ensure that all devices were available during the entire training in practice. It could not guarantee that devices would complete their assigned training tasks in every training round as expected. Specifically, the research incorporated four situations: in-completeness where devices submitted only partially completed work in a round, inactivity where devices did not complete any updates or respond to the coordinator at all, early departures where existing devices quit the training without finishing all training rounds, and late arrivals where new devices joined after the training has already started.

In [89], a **Federated Autonomous Driving network (FADNet)** was designed to improve FL model stability, ensure convergence, and handle imbalanced data distribution problems. The experiments were conducted with a dense topology called the Internet Topology Zoo (Gaia) [53].

A federated learning on a fully decentralized network problem was analyzed in [48], particularly on how the convergence of a decentralized FL system will be affected under different network settings. Several simulations were conducted with different topologies, datasets, and machine learning models. The end results suggested that scale-free and small-world networks are more suitable for decentralized FL and a hierarchical network has convergence speed with trade-offs.

4.3.3 Blockchain-Based Topology. As one of the decentralized methods for FL, numerous blockchain-based topologies have been presented in previous works. The blockchains combined with FL aim at replacing the central server for generating the global model. Figure 9 displays a typical blockchain-based FL topology. The highlighted works of blockchain-based topology are shown in Table 7. The potential benefits of introducing blockchains in FL systems include the following:

- Placement of model training
- Incentive mechanism to attract more participants
- Decentralized privacy
- Defending poison attacks
- Cross verification

Table 7. Highlighted Works - Blockchain Topology

FL Type	Baselines and Benchmarks	Key Findings
Blockchain	Basic FL and stand-alone training framework with FEMNIST	Higher resistance to malicious nodes, mitigate the influence of malicious central servers or nodes [65]
	FL-Block with CIFAR-10, FASHION-MINIST	Fully capable to support big data scenarios, particularly fog computing applications, provides decentralized privacy protection while preventing a single point of failure [103]
	Leaf with Ethereum as the underlying blockchain, tested logistic regression (LR) and NNs models	Incentive mechanisms encouraged clients to provide high-quality training data, communication overhead can be significantly reduced when the data set size is extremely large [158]
	Integrate FL in consensus process of permissioned blockchain with Reuters and 20 newsgroups dataset	Increased efficiency of data sharing scheme by improving utilization of computing resources; secure data sharing with high utility and efficiency [76]
	Provided assistance to home appliance manufacturers using FL to predict future customer demands and behavior with MNIST	Created an incentive program to reward participants while preventing poisoning attacks from malicious customers; communication costs are small compared with wasted training time on mobile [160]
	Evaluation were based on 3GPP LTE Cat. M1 specification	Allowed autonomous vehicles to communicate efficiently, as it exploited consensus mechanisms in blockchain to enable oVML ML without centralized server [100]
	FL with Multi-Krum and DP under position attacks using Credit Card dataset and MNIST	Scalable; fault-tolerant; defend against known attacks; capable of protecting the privacy of client updates and maintaining the performance of the global model with 30% adversaries [114]

FLChain was proposed in [80] to enhance the reliability of FL in wireless networks for separate channel selection used by FL model uploading and downloading, where local model parameters were stored as blocks on a blockchain as an alternative to a central aggregation server and the edge devices provided network resources to the resource-constraint mobile devices and served as nodes in the blockchain network of FLChain. Similar to most blockchain-based FL frameworks, FLChain had the blockchain network above the edge devices for channel registration and global model updates.

A blockchain-based federated learning framework with committee consensus (BFLC) was proposed in [65] to reduce the amount of consensus computing and malicious attacks. The alliance blockchain was used to manage FL nodes for permission control. Different from the traditional FL process, there is an additional committee between the training nodes and the central server for updates selection. In each round of FL, updates were validated and packaged by the selected committee, allowing the most honest nodes to improve the global model continuously. A small number of incorrect or malicious node updates will be ignored to avoid damaging the global model. Nodes can join or leave at any time without damaging the training process. The blockchain acted as a distributed storage system for persisting the updates.

Qu et al. developed FL-Block [103] to allow the exchange of local learning updates from end devices via a blockchain-based global learning model verified by miners. The central authority was replaced with an efficient blockchain-based protocol. The blockchain miners verified and stored the local model updates. A linear regression problem was presented with the objective of minimizing a loss function $f(\omega)$. An algorithm designed for block-enabled FL enabled block generation by the winning miner after the local model was uploaded to the fog servers. The fog servers received updates of global models from the blockchain.

A blockchain anchoring protocol was designed [158] for device failure detection. Specifically, a blockchain anchoring protocol was designed which built custom Merkle trees with each leaf node representing a record of data root onto blockchains to verify **Industrial IoT (IIoT)** data integrity efficiently.

In a similar research scenario of processing IIoT data, a permissioned blockchain [101] was used in [76] for recording IIoT data retrieval and data sharing transactions. The **Proof of Training Quality (PoQ)** was proposed to replace the original **Proof of Work (PoW)** mechanism for lower cost reaching consensus. A differential privacy preserved model was first incorporated into FL. Regarding the PoQ, the committee leader was selected according to the trained model by prediction accuracy, measured by the **mean absolute errors (MAEs)**:

$$MAE(m_i) = \frac{1}{n} \sum_{i=1}^n |y_i - f(x_i)|, \quad (4)$$

where $f(x_i)$ denoted the prediction value of model m_i and y_i was the observed value. The consensus process started with the election of the committee leader with the lowest MAE^u by voting. This leader was then assigned to drive the consensus process. The trained models were circulated among the neighboring committee nodes, denoted by P_i , of a committee node P_j , leading to the MAE for P_j to be

$$MAE^u(P_j) = \gamma \cdot MAE(m_j) + \frac{1}{n} \sum_{i=1}^n MAE(m_i), \quad (5)$$

where $MAE(m_j)$ was the locally trained model weighted by γ , and $MAE(m_i)$ referred to remotely trained models.

Zhao et al. [160] replaced aggregator nodes in traditional FL systems with blockchains for traceable activities. Customer data was selected to be sent to selected miners for averaging. One of the miners, selected as the leader, uploaded the aggregated model to the blockchain. More importantly, the authors proposed a normalization technique with differential privacy preservation.

Pokhrel and Choi [100] discussed **blockchain-based FL (BFL)** parameters for vehicular communication networking, considering local on-vehicle machine learning updates. The blockchain-related parameters were discussed via a mathematical framework, including the retransmission rate, block size, block arrival rate, block arrival rate, and frame sizes. The analytical results proved that tuning the block arrival rate was able to minimize the system delay.

Shayan et al. proposed a fully decentralized multi-party ML (Bitscotti) [114] using blockchain and cryptographic emphasis on privacy-preserving. The training process of clients is stored in the blockchain ledger. Clients complete local training and the results are masked using a private noise. Then the masked updates go through a validation process as an extra layer of security. A new block is created for every new round of training. However, due to communication overhead, Bitscotti does not support large deep learning models. With a size of 200 peers, the test shows similar utility compared to traditional star topology federated learning.

4.4 Minor Topologies

In addition to the above-mentioned topologies, some minor topologies combine existing topologies or utilize niche new topologies that are not widely used. Although there are only a few studies on some minor topologies, these works can still provide valuable insight into the subject. The highlighted works of minor topology are shown in Table 8.

4.4.1 Ring Topology. A **ring-topology decentralized federated learning (RDFL)** framework was proposed in [137] for communication-efficient learning across multiple data sources in a decentralized environment. RDFL was inspired by the idea of ring-allreduce³ and applied a consistent hashing technique to construct a ring topology of decentralized nodes. An IPFS-based data-sharing scheme was designed as well to reduce communication costs.

³<https://andrew.gibiansky.com/blog/machine-learning/baidu-allreduce/>

Table 8. Highlighted Works - Minor Topologies

FL Type	Baselines and Benchmarks	Key Findings
Ring	FedAvg with MNIST, CIFAR-10 and CIFAR-100	Improved bandwidth utilization, robustness, and system communication efficiency, reduce communication costs [137]
	G-plain (Graph-based), R-plain (ring-based), and UBAR with MNIST and CIFAR-10	Fast and computationally efficient; superior performance with SOTA in IID and non-IID; achieve linear convergence rate; further scalability in parallel implementation [25]
	LeNet and VGG11 with MNIST, FMNIST, EMNIST, CIFAR-10	Achieved higher test accuracy in fewer communication rounds; faster convergence, robustness to non-IID dataset [144]
Clique	Fedavg with MNIST and CIFAR10	Reduced gradient bias, convergence in heterogeneous data environments, reduction in edge and message numbers [4]
Fog	FedAvg, HierFAVG, DPSGD with MNIST, FEMNIST, Synthetic dataset	Robust under dynamic topologies; fastest convergence rates under both static and dynamic topologies [161]
	FedAvg with MNIST, FEMNIST, Shakespeare	Gave smooth convergence curve; higher model accuracy; more scalable; communication-efficient [17]
	FL with full device participation and FL with one device sampled from each cluster with MNIST, F-MNIST	Better model accuracy; energy consumption; robustness against outages; favorable performance with non-convex loss functions [69]
	Only Cloud, INC Solution, Non-INC, and INC LB	Reached near-optimal network latency; outperformed baselines; helped cloud node significantly decrease its network's aggregation latency, traffic, and computing load [22]
	FedAsync and FedAvg with MNIST and CIFAR-10	Reduced consumption of network traffic; faster converges; effective with non-IID data; dealing with staleness [138]
Semi-ring	FedAvg with MNIST under non-IID setting	Improved communication efficiency, flexible and adaptive convergence [125]
	Astraea, FedAvg, HierFavg, IFCA, MM-PSGD, SemiCyclic with FedShakespeare, MNIST	Near-linear scalability; improved model accuracy [59]

RingFed [144] took advantage of the ring topology setup allowing clients to communicate with each other while performing preaggregation on clients to further reduce communication rounds. RingFed does not rely on the central server to perform model training tasks but uses the central server to assist in the passing of model parameters. The client only communicated with the central server when the set number of periods. In comparison to other algorithms, an additional step of recalculating all client parameters is added. Experimental results show that RingFed outperforms FedAvg in most cases and the results of training are optimized on non-IID data as well.

Elkordy et al. [25] proposed Basil, a fast and computationally efficient Byzantine robust algorithm for decentralized (serverless) training systems. In particular, the key aspect of their work is that it considers the decentralized FL and leverages the logical ring topology among nodes. Basil has also proven to achieve a linear convergence rate and further scalability in parallel implementation.

4.4.2 Clique Topology. Cliques are defined in graph theory, referring to a subset of vertices of an undirected graph such that every two distinct vertices in the clique are adjacent [2]. Cliques have been well-studied in graph theory. Cliques have also been used in FL for improving the accuracy in sparse neural networks [4]. We used the clique-based topology structure from Bellet et al. [4] as a visualized example shown in Figure 10.

D-cliques [4] was a topology that reduced the gradient bias by grouping nodes in sparsely interconnected cliques such that the label distribution in a clique is representative of the global label

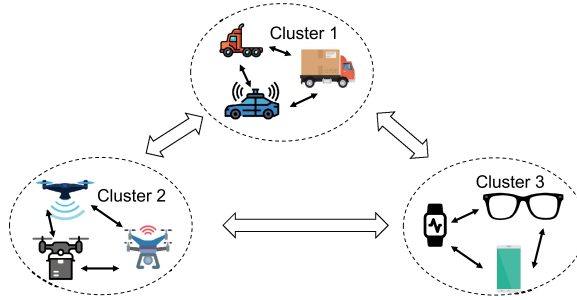


Fig. 10. The clique-based topology structured in D-cliques [4] that could affect the structure of underlying topologies.

distribution. This way, the impact of label distribution skew can be mitigated for heterogeneous data. Instead of providing a fully connected topology which may be unrealistic with large numbers of clients, D-Cliques instead provided locally fully connected neighborhoods. Each node belonged to a Clique, a set of fully connected nodes with data distribution as close as possible to the global distribution of the data through the network. Each Clique of the network provided a fair representation of the true data distribution, while substantially reducing the number of links.

4.4.3 Grid Topology. The grid topology also enables data transmission between adjacent clients. Compared to the ring topology, where each client has two one-hop neighbors, the grid topology gives each client four neighbors, encouraging more data exchange in local networks. Shi et al. [117] discussed a scenario of distributed federated learning in a multi-hop wireless network. The experiments evaluated the performance over the line, ring, star, and grid networks, proving that more neighbors would lead to faster convergence and higher accuracy.

4.4.4 Hybrid Topology. In the previous section, we have introduced various types of topologies frequently seen in prior FL studies. Though these topologies cover a great portion of the use cases, each topology has its pros and cons. Researchers have explored combinations of various topologies to receive maximum benefits from network topologies. Hybrid topologies [17, 39, 59, 125] combine the strengths of at least two traditional topologies to create a more dynamic solution.

4.4.5 Fog Topology (Star + Mesh). In this section, we examine the fog topology, which is essentially a fusion of star and mesh topologies. We show a visualization of fog topology from the works of Hosseinalipour et al. [39] shown in Figure 11.

The concept of fog learning was presented in [39] compared to FL over heterogeneous wireless networks. The word “fog” was used to address the heterogeneity across devices. Compared to FL, fog learning considers the diversity of devices with various proximities and topology structures for scalability. The proposed fog learning boasted its multi-layer network architecture and its vertical and horizontal device communications ability. Device-to-device (D2D) communications were possible when there were fewer privacy concerns. Compared to the tree topology, additional D2D communication paths were added at the edge layer. The D2D offloading could happen among trusted devices, at the cost of privacy compromise, if such a sacrifice were acceptable. Inter-layer data offloading could also be implemented to increase the similarity of local data and reduce model bias.

Hosseinalipour et al. [38] developed **multi-stage hybrid federated learning (MH-FL)** built on fog learning which is a hybrid of intra- and inter-layer model learning that considered the network as a multi-layer, hybrid structure with both mesh topology and tree topology. In MH-FL,

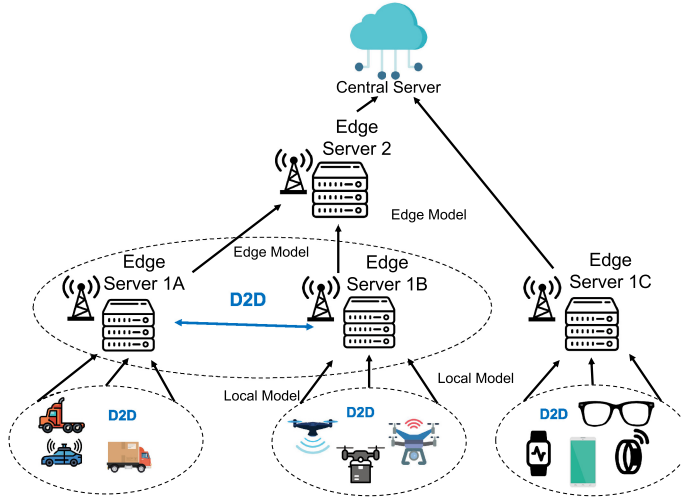


Fig. 11. The fog learning topology [39], where D2D communications are enabled among the clients in the same cluster, as well as edge servers in the same cluster.

each layer of which consists of multiple device clusters. MH-FL considered the topology structures among the nodes in the clusters, including local networks formed via D2D communications. It orchestrated the devices at different network layers in a collaborative/cooperative manner to form a local consensus on the model parameters and combined it with multi-stage parameters relaying between layers of the tree-shaped hierarchy. These clusters were designed in two types: **limited uplink transmission (LUT)** clusters with limited capability to upload data to the upper layer, and **extensive uplink transmission (EUT)** clusters with enough resources to perform conventional FL.

Strictly speaking, some topologies are based on tree topologies but with additional edges [161], making them more genetic graphs with more connectivity. To scale up FL, **Parallel FL (PFL)** systems were built with multiple **parameter servers (PS)**. A parallel FL algorithm called P-FedAvg was proposed in [161], extending FedAvg by allowing multiple parameter servers to work together. The authors identified that a single parameter server became the bottleneck due to two reasons: the difficulty in establishing a fast network that connects all devices to a single PS, as well as the limited communication capacity of only one PS. With the P-FedAvg algorithm, each client conducted several local iterations before uploading the model parameters to its PS. A PS collected model parameters from selected clients and conducted a global iteration by aggregating the model parameters uploaded from its clients and then mixing model parameters with its neighbor PS. The authors optimized the weights for PS to mix their parameters with neighbors. Essentially, this was a non-global aggregation without requiring communications with the central server. The study indicated that PFL could significantly improve the convergence rate if the network was not sparsely connected. They also compared its P-FedAvg under three different network topologies: Ring, 2d-torus, and Star, while 2d-torus is the most robust.

FedP2P was proposed in [17], aiming at reorganizing the connectivity structure to distribute both the training and communication on the edge devices by leveraging P2P communication. While edge devices performed pairwise communication in a D2D manner, a central server was still in place. However, the central server only communicated with a small number of devices. Each of these small numbers of devices represented the partition. The parameters had been aggregated before being transmitted to the central server. Compared to the tree-based topology, FedP2P

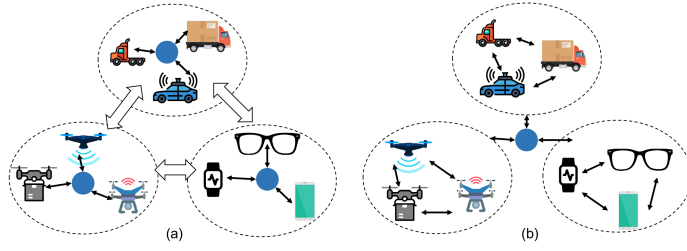


Fig. 12. The semi-ring topology described in [59].

was robust if one or more nodes in a P2P subnetwork were down. Compared to the original star topology, FedP2P still had better scalability with clustered P2P networks. Lin et al. proposed a semi-decentralized learning architecture called TT-HF [69], which combined the traditional star topology of FL with decentralized D2D communications for model training, formulating a *semi-decentralized* FL topology. The problem of resource-efficient federated learning across heterogeneous local datasets at the wireless edge was studied. D2D communications were enabled. A consensus mechanism to mitigate model divergence was developed to mitigate low-power communications among nearby devices. TT-HF incorporated two timescales for model training, including iterations of stochastic gradient descent at individual devices and rounds of cooperative D2D communications within clusters.

Dinh et al. [22] proposed an edge network architecture that decentralized the model aggregation process at the server and significantly reduced the aggregation latency. First, an in-network aggregation process was designed so that the majority of aggregation computations were offloaded from the cloud server to edge nodes. Then a joint routing and resource allocation optimization problem was formulated to minimize the aggregation latency for the whole system at every learning round. Numerical results showed a 4.6 times improvement in the network latency. FedCH [138] constructed a special cluster topology and performed hierarchical aggregation for training. FedCH arranged clients into multiple clusters based on their heterogeneous training capacities. The cluster head collected all updates from clients in that cluster for aggregation. All cluster headers took the asynchronous method for global aggregation. The authors concluded that the convergence bound was related to the number of clusters and the training epochs, and then proposed an algorithm for the optimal number of clusters with resource budgets and with the cluster topology, showing an improvement of completion time by 49.5–79.5% and the network traffic by 57.4–80.8%.

4.4.6 Semi-Ring Topology (Ring + Star/Tree). In addition to comprehensively comparing FL system structures with different topologies, the ring and tree topology were used in [125] by Tao et al. for efficient parameter aggregation. A hybrid network topology design was proposed integrating ring (R) and n-ary tree (T) to provide flexible and adaptive convergencast in federated learning. Participating peers within one-hop were formed as a local ring to adapt to the frequent joining and leaving of devices; an n-ary convergencast tree was formed from local rings to the aggregator for communication efficiency. Theoretical analysis found that the hybrid (R+T) convergencast design was superior for system latency. We show these hybrid topologies, termed as semi-ring Topology in Figure 12.

Lee et al. presented an algorithm called TornadoAggregate [59] by facilitating the ring architecture to improve the accuracy and scalability of FL. A global inter-node transfer model that is synchronized with the new model will replace traditional global aggregation in the traditional star architecture. TornadoAggregate can achieve a low convergence bound and satisfy the diurnal property condition.

5 CHALLENGES AND FUTURE RESEARCH ROADMAPS

Despite great attention addressing topology-related challenges for edge computing FL in recent years [4, 15, 84, 137], the nature of the network topologies and data distribution still introduces unique challenges. Apart from the previously mentioned topology-aware FL works, much is still to be studied about network topology in FL. In this section, we provide some open challenges and research directions for topology-aware FL.

5.1 Topology Selection

When implementing or designing a topology-aware FL approach, there are a few things to consider. For example, in hierarchical and heterogeneous edge networks, multiple paths exist from the edge devices to the edge servers and the central server. When selecting the topology optimal for FL, the following questions need to be asked:

- (1) **Does a server-less architecture fit the system?**
- (2) **Is the traditional star topology no longer sufficient for the system?**
- (3) **Is there a unique topology that already exists at the hardware or structural level in the system?**

When a system structure already exists, for example, a network of devices and subgroups of those devices controlled and managed by an intermediate server, the tree topology structure will be an obvious choice. As a result, the focus will shift from which topology to select to how to optimize the tree topology structure for a specific goal, i.e., for increased communication efficiency or to mitigate security bottlenecks. This area offers many opportunities for further research, including new topologies development or combinations of existing topologies.

5.2 Communication Cost

It is common for edge devices to be powered by batteries. Performing local model aggregation and wireless model transmission consume the limited power sources of edge devices. Saving communication costs and developing energy-aware federated algorithms are aligned with our primary goals. Moreover, existing solutions save communication costs from the amount of data transmissions. Further research can be conducted on topology control algorithms that can optimize energy consumption in conjunction with network topology.

The network heterogeneity and ever-changing nature of edge devices pose great challenges to FL. The heterogeneity of the edge networks determines that the bandwidth resources vary for links in edge networks. Meanwhile, the mobility and density of the edge devices further reduce the actual bandwidth of those links. In the worst case, certain links in the edge networks suffer connectivity loss. The changing link conditions in edge networks demand dynamic and fault-tolerant network topologies for FL to aggregate data. Based on the amount of data for model transmission, models and algorithms are needed to find the most effective topologies to deliver the model reliably on time.

5.3 Client-Drift

Due to the large number of edge devices and statistical heterogeneity, a phenomenon known as client drift [47] could occur. Client drift occurs when clients with non-IID data develop extremely distinct local models away from the global optimal model. Some clients can be seen as noisy since their updates can be misleading global models. Some edge devices may become particularly “noisy” and their local model updates can dominate the global model weights. The problem is even more severe in the clusters with one noisy edge device. If FL leverages the cluster’s local model for FL, it may be too biased towards the models by the noisy nodes. A solution to mitigate such a

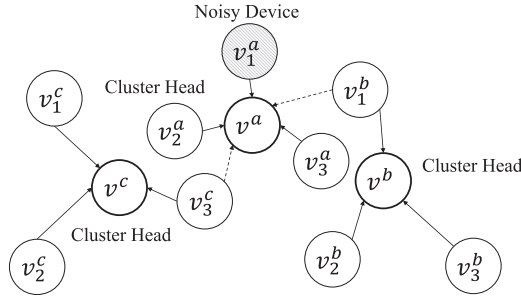


Fig. 13. Topology control for cluster-based model fairness with noisy edge devices.

biased model at the cluster heads or edge servers can be an opportunistic routing that intentionally integrates models from edge devices outside a cluster. For our example in the left of Figure 13, the local models learned can be contributed to multiple clusters. The slicing of the data reduces the exposure of repeated model transmissions updated on the same dataset to the same edge servers and therefore enhances privacy.

5.4 Ethical/Privacy Concerns

In this section, we discuss new ethical and privacy challenges possessed by different network topology structures. The primary concern with the standard star topology is the communication bottleneck and excessive reliance on a central server. The central server is heavily tasked with safeguarding all client information. The default star topology in FL can be represented as a single point of failure potentially compromising the privacy of the entire network and raising potential ethical concerns. Other network structures can address some of the privacy concerns of the traditional topology structure. For tree topology, the additional communication layers and intermediate servers present both advantages and disadvantages. The tree topology allows the central server to offload some computational tasks and client information to the intermediate server. However, the presence of the intermediate servers requires greater efforts towards privacy on the edge. Other fully decentralized topologies do not require a central server, such as mesh, gossip, clique, and grid topology. Without an overarching central server, there is no need for direct communication with the server, which would normally pose a significant privacy threat and ethical violations. What comes with this is the increased amount of peer-to-peer (P2P) communication which could introduce new privacy challenges.

The extended period of model aggregations, “devices $\rightarrow$ edge servers $\rightarrow$ central server”, could lead to privacy concerns regarding the training data of the edge devices. When an edge device sends its aggregated model upon a global model update request, the model will be broadcast to its neighbors. Repeated rounds of sharing models among the nearby neighbors will mix the device’s local model with the sub-local models. The changes in the topology select changing sets of neighbors, further increasing the diversity of local models. The model mingling activities will reduce the vulnerability of devices suffering differential privacy attacks.

In conclusion, privacy is always a trade-off. No single topology can meet all the needs of all users. As technology advances, the existing topology will face new challenges. New and niche topologies will present new challenges and opportunities. Various aspects of network topology and their impact on ethical and privacy issues require further research.

5.5 Availability-Aware FL Assisted by Topology Overlay

On top of that, the model aggregation tasks at the edge servers, also known as cluster heads in a hierarchical FL architecture, face availability challenges when the edge servers are down. While

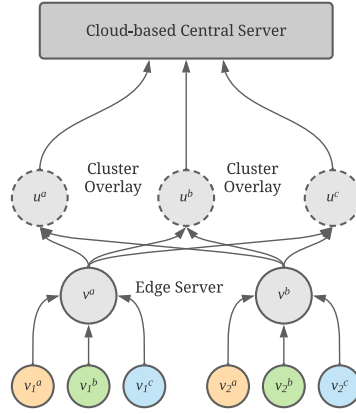


Fig. 14. An overlay network on top of edge computing clusters.

with the meshed links among the edge servers, learning task replication techniques can be used for maintaining the availability levels of the model aggregation tasks. In other words, we must make a trade-off between robustness and redundancy. The problem can be further investigated from a resource allocation perspective, scheduling, and clustering. In the right of Figure 14, the clusters can be built logically instead of following the physical topology of the edge networks based on the intensity and the distribution of data generation. In the example, there are two physical clusters v^a and v^b . The three edge devices in a cluster belong to one of the three separate overlay clusters u^a , u^b , and u^c .

5.6 Conduct Real-World Deployment

In most topology-related FL works, their experiments are conducted in simulated environments, with the exceptions of [22, 31, 50, 100, 126, 127, 135, 162]. Within these works, Zhou et al. [162] only used the Alibaba Cloud as its parameter server and Tran and Pompili [127] simply designed their experiment using realistic model settings from [18]. The experiment setting for [22] although still simulations, does involve real deployment on a grid network inside a $500\text{m} \times 500\text{m}$ area. Wang et al. [135] use a unique approach that captures the *Xender's* trace content and requests files from active mobile users. Many FL works to date do not include or consider real-world model deployments in their experiments. However, this type of work can demonstrate real challenges faced when deploying unique edge topologies in a realistic setting while tackling specific issues such as model deployment time, inference time, communication costs, and so on. For further research, it would be advantageous to conduct experiments with real-world deployment and evaluate topology-aware FL studies in real edge environments. This approach ensures the proposed FL techniques and algorithms can be properly validated, moving beyond just proofs-of-concept in simulated environments. For topology-aware FL, much work can be done to develop a realistic test bed and to perform real-world deployments.

6 CONCLUSION

In this survey, the role of topology-aware federated learning in edge computing is discussed in detail. Various network topologies, including star, tree, decentralized, and hybrid topologies, are summarized and compared to illustrate the substantial impact of topology on the efficiency and effectiveness of federated learning. Different topologies can bring many benefits to the network. It is important to note that various topology structures will undoubtedly bring extra complexity.

There is a choice to be made if the simple star topology cannot meet the needs of a growing system and infrastructure and whether to opt for another topology for increased communication and complexity. FL architectures must also account for factors such as the central server's necessity or absence, clients' diurnal activity patterns, and options for implementing intermediate servers.

REFERENCES

- [1] Ali Al-Shuwaili and Osvaldo Simeone. 2017. Energy-efficient resource allocation for mobile edge computing-based augmented reality applications. *IEEE Wireless Communications Letters* 6, 3 (2017), 398–401.
- [2] Richard D. Alba. 1973. A graph-theoretic definition of a sociometric clique. *Journal of Mathematical Sociology* 3, 1 (1973), 113–126.
- [3] James Henry Bell, Kallista A. Bonawitz, Adrià Gascón, Tancrede Lepoint, and Mariana Raykova. 2020. Secure single-server aggregation with (poly) logarithmic overhead. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 1253–1269.
- [4] Aurélien Bellet, Anne-Marie Kermarrec, and Erick Lavoie. 2021. D-Cliques: Compensating nonIIDness in decentralized federated learning with topology. *arXiv preprint arXiv:2104.07365* (2021).
- [5] Keith Bonawitz, Hubert Eichner, Wolfgang Grieskamp, Dzmitry Huba, Alex Ingerman, Vladimir Ivanov, Chloe Kid-don, Jakub Konečný, Stefano Mazzocchi, Brendan McMahan, Timon Van Overveldt, David Petrou, Daniel Ramage, and Jason Roselander. 2019. Towards federated learning at scale: System design. *Proceedings of Machine Learning and Systems* 1 (2019), 374–388.
- [6] Keith Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. 2017. Practical secure aggregation for privacy-preserving machine learning. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. 1175–1191.
- [7] Christopher Briggs, Zhong Fan, and Peter Andras. 2020. Federated learning with hierarchical clustering of local updates to improve training on non-IID data. In *2020 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–9.
- [8] Qiming Cao, Xing Zhang, Yushun Zhang, and Yongdong Zhu. 2021. Layered model aggregation based federated learning in mobile edge networks. In *2021 IEEE/CIC International Conference on Communications in China (ICCC)*. IEEE, 1–6.
- [9] Zheng Chai, Ahsan Ali, Syed Zawad, Stacey Truex, Ali Anwar, Nathalie Baracaldo, Yi Zhou, Heiko Ludwig, Feng Yan, and Yue Cheng. 2020. TiFL: A tier-based federated learning system. In *Proceedings of the 29th International Symposium on High-Performance Parallel and Distributed Computing*. 125–136.
- [10] Zheng Chai, Yujing Chen, Liang Zhao, Yue Cheng, and Huzefa Rangwala. 2020. FedAT: A communication-efficient federated learning method with asynchronous tiers under non-IID data. *ArXivorg* (2020).
- [11] Zachary Charles, Zachary Garrett, Zhouyuan Huo, Sergei Shmulyan, and Virginia Smith. 2021. On large-cohort training for federated learning. *Advances in Neural Information Processing Systems* 34 (2021).
- [12] Daoyuan Chen, Liuyi Yao, Dawei Gao, Bolin Ding, and Yaliang Li. 2023. Efficient personalized federated learning via sparse model-adaptation. *arXiv preprint arXiv:2305.02776* (2023).
- [13] Min Chen and Yixue Hao. 2018. Task offloading for mobile edge computing in software defined ultra-dense network. *IEEE Journal on Selected Areas in Communications* 36, 3 (2018), 587–597.
- [14] Zhuo Chen, Wenlu Hu, Junjue Wang, Siyan Zhao, Brandon Amos, Guanhang Wu, Kiryong Ha, Khalid Elgazzar, Padmanabhan Pillai, Roberta Klatzky, Daniel Siewiorek, and Mahadev Satyanarayanan. 2017. An empirical study of latency in an emerging class of edge computing applications for wearable cognitive assistance. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*. 1–14.
- [15] Zhikun Chen, Daofeng Li, Jinkang Zhu, and Sihai Zhang. 2021. DACFL: Dynamic average consensus based federated learning in decentralized topology. *arXiv preprint arXiv:2111.05505* (2021).
- [16] Beongjun Choi, Jy-yong Sohn, Dong-Jun Han, and Jaekyun Moon. 2020. Communication-computation efficient secure aggregation for federated learning. *arXiv preprint arXiv:2012.05433* (2020).
- [17] Li Chou, Zichang Liu, Zhuang Wang, and Anshumali Shrivastava. 2021. Efficient and less centralized federated learning. *arXiv preprint arXiv:2106.06627* (2021).
- [18] Xiaoli Chu, David Lopez-Perez, Yang Yang, and Fredrik Gunnarsson. 2013. *Heterogeneous Cellular Networks: Theory, Simulation and Deployment*. Cambridge University Press.
- [19] Peijin Cong, Junlong Zhou, Liying Li, Kun Cao, Tongquan Wei, and Keqin Li. 2020. A survey of hierarchical energy optimization for mobile edge computing: A perspective from end devices to the cloud. *ACM Computing Surveys (CSUR)* 53, 2 (2020), 1–44.
- [20] Yongheng Deng, Feng Lyu, Ju Ren, Yongmin Zhang, Yuezhi Zhou, Yaoxue Zhang, and Yuanyuan Yang. 2021. SHARE: Shaping data distribution at edge for communication-efficient hierarchical federated learning. In *2021 IEEE 41st International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 24–34.

- [21] Jie Ding, Eric Tramel, Anit Kumar Sahu, Shuang Wu, Salman Avestimehr, and Tao Zhang. 2022. Federated learning challenges and opportunities: An outlook. In *ICASSP 2022*. <https://www.amazon.science/publications/federated-learning-challenges-and-opportunities-an-outlook>
- [22] Thanh Quang Dinh, Diep N. Nguyen, Dinh Thai Hoang, Pham Tran Vu, and Eryk Dutkiewicz. 2021. Enabling large-scale federated learning over wireless edge networks. *arXiv preprint arXiv:2109.10489* (2021).
- [23] Benoit Donnet and Timur Friedman. 2007. Internet topology discovery: A survey. *IEEE Communications Surveys & Tutorials* 9, 4 (2007), 56–69.
- [24] Moming Duan, Duo Liu, Xianzhang Chen, Renping Liu, Yujuan Tan, and Liang Liang. 2020. Self-balancing federated learning with global imbalanced data in mobile systems. *IEEE Transactions on Parallel and Distributed Systems* 32, 1 (2020), 59–71.
- [25] Ahmed Roushdy Elkordy, Saurav Prakash, and Salman Avestimehr. 2022. Basil: A fast and Byzantine-resilient approach for decentralized training. *IEEE Journal on Selected Areas in Communications* 40, 9 (2022), 2694–2716.
- [26] Hossein Fereidooni, Samuel Marchal, Markus Miettinen, Azalia Mirhoseini, Helen Möllering, Thien Duc Nguyen, Phillip Rieger, Ahmad-Reza Sadeghi, Thomas Schneider, Hossein Yalame, et al. 2021. SAFELearn: Secure aggregation for private Federated learning. In *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, 56–62.
- [27] Anousheh Gholami, Nariman Torkzaban, and John S. Baras. 2022. Trusted decentralized federated learning. In *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 1–6.
- [28] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. 2017. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677* (2017).
- [29] Yinghao Guo, Rui Zhao, Shiwei Lai, Lisheng Fan, Xianfu Lei, and George K. Karagiannidis. 2022. Distributed machine learning for multiuser mobile edge computing systems. *IEEE Journal of Selected Topics in Signal Processing* (2022).
- [30] Otkrist Gupta and Ramesh Raskar. 2018. Distributed learning of deep neural network over multiple agents. *Journal of Network and Computer Applications* 116 (2018), 1–8.
- [31] Andrew Hard, Kanishka Rao, Rajiv Mathews, Swaroop Ramaswamy, Françoise Beaufays, Sean Augenstein, Hubert Eichner, Chloé Kiddon, and Daniel Ramage. 2018. Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604* (2018).
- [32] Chaoyang He, Emir Ceyani, Keshav Balasubramanian, Murali Annavam, and Salman Avestimehr. 2021. Spread-GNN: Serverless multi-task federated learning for graph neural networks. *arXiv preprint arXiv:2106.02743* (2021).
- [33] Chaoyang He, Conghui Tan, Hanlin Tang, Shuang Qiu, and Ji Liu. 2019. Central server free federated learning over single-sided trust social networks. *arXiv preprint arXiv:1910.04956* (2019).
- [34] Ziqi He, Lei Yang, Wanyu Lin, and Weigang Wu. 2022. Improving accuracy and convergence in group-based federated learning on non-IID data. *IEEE Transactions on Network Science and Engineering* (2022).
- [35] István Hegedűs, Árpád Berta, Levente Kocsis, András A. Benczúr, and Márk Jelasity. 2016. Robust decentralized low-rank matrix decomposition. *ACM Transactions on Intelligent Systems and Technology (TIST)* 7, 4 (2016), 1–24.
- [36] István Hegedűs, Gábor Danner, and Márk Jelasity. 2019. Gossip learning as a decentralized alternative to federated learning. In *IFIP International Conference on Distributed Applications and Interoperable Systems*. Springer, 74–90.
- [37] Samuel Horvath, Stefanos Laskaridis, Mario Almeida, Ilias Leontiadis, Stylianos Venieris, and Nicholas Lane. 2021. Fjord: Fair and accurate federated learning under heterogeneous targets with ordered dropout. *Advances in Neural Information Processing Systems* 34 (2021), 12876–12889.
- [38] Seyyedali Hosseinalipour, Sheikh Shams Azam, Christopher G. Brinton, Nicolo Michelusi, Vaneet Aggarwal, David J. Love, and Huaiyu Dai. 2020. Multi-stage hybrid federated learning over large-scale D2D-enabled fog networks. *arXiv preprint arXiv:2007.09511* (2020).
- [39] Seyyedali Hosseinalipour, Christopher G. Brinton, Vaneet Aggarwal, Huaiyu Dai, and Mung Chiang. 2020. From federated to fog learning: Distributed machine learning over heterogeneous wireless networks. *IEEE Communications Magazine* 58, 12 (2020), 41–47.
- [40] Chenghao Hu, Jingyan Jiang, and Zhi Wang. 2019. Decentralized federated learning: A segmented gossip approach. *arXiv preprint arXiv:1908.07782* (2019).
- [41] Erdong Hu, Yuxin Tang, Anastasios Kyrillidis, and Chris Jermaine. 2023. Federated learning over images: Vertical decompositions and pre-trained backbones are difficult to beat. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 19385–19396.
- [42] Shanfeng Huang, Shuai Wang, Rui Wang, and Kaibin Huang. 2021. Joint topology and computation resource optimization for federated edge learning. In *2021 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 1–6.
- [43] Congfeng Jiang, Tiantian Fan, Honghao Gao, Weisong Shi, Liangkai Liu, Christophe Cerin, and Jian Wan. 2020. Energy aware edge computing: A survey. *Computer Communications* 151 (2020), 556–580.
- [44] Jingyan Jiang and Liang Hu. 2020. Decentralised federated learning with adaptive partial gradient aggregation. *CAAI Transactions on Intelligence Technology* 5, 3 (2020), 230–236.

- [45] Jingyan Jiang, Liang Hu, Chenghao Hu, Jiatae Liu, and Zhi Wang. 2020. BACombo-bandwidth-aware decentralized federated learning. *Electronics* 9, 3 (2020), 440.
- [46] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D'Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badi Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaid Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrede Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Hang Qi, Daniel Ramage, Ramesh Raskar, Mariana Raykova, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. 2021. Advances and open problems in federated learning. *Foundations and Trends in Machine Learning* 14, 1–2 (2021), 1–210.
- [47] Sai Praneeth Karimireddy, Satyen Kale, Mehryar Mohri, Sashank Reddi, Sebastian Stich, and Ananda Theertha Suresh. 2020. Scaffold: Stochastic controlled averaging for federated learning. In *International Conference on Machine Learning*. PMLR, 5132–5143.
- [48] Hanna Kavalionak, Emanuele Carlini, Patrizio Dazzi, Luca Ferrucci, Matteo Mordacchini, and Massimo Coppola. 2021. Impact of network topology on the convergence of decentralized federated learning systems. In *2021 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 1–6.
- [49] Latif U. Khan, Walid Saad, Zhu Han, Ekram Hossain, and Choong Seon Hong. 2021. Federated learning for internet of things: Recent advances, taxonomy, and open challenges. *IEEE Communications Surveys & Tutorials* (2021).
- [50] Wasiq Khan, Abir Hussain, Bilal Muhammad Khan, and Keeley Crockett. 2023. Outdoor mobility aid for people with visual impairment: Obstacle detection and responsive framework for the scene perception during the outdoor mobility of people with visual impairment. *Expert Systems with Applications* 228 (2023), 120464.
- [51] Fahad Ahmed KhoKhar, Jamal Hussain Shah, Muhammad Attique Khan, Muhammad Sharif, Usman Tariq, and Seifedine Kadry. 2022. A review on federated learning towards image processing. *Computers & Electrical Engineering* 99 (2022), 107818.
- [52] Abbas Kiani and Nirwan Ansari. 2017. Toward hierarchical mobile edge computing: An auction-based profit maximization approach. *IEEE Internet of Things Journal* 4, 6 (2017), 2082–2091.
- [53] Simon Knight, Hung X. Nguyen, Nickolas Falkner, Rhys Bowden, and Matthew Roughan. 2011. The internet topology zoo. *IEEE Journal on Selected Areas in Communications* 29, 9 (2011), 1765–1775.
- [54] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. 2016. Federated learning: Strategies for improving communication efficiency. *arXiv preprint arXiv:1610.05492* (2016).
- [55] Nicolas Kourtellis, Kleomenis Katevas, and Diego Perino. 2020. FLaaS: Federated learning as a service. In *Proceedings of the 1st Workshop on Distributed Machine Learning*. 7–13.
- [56] Alex Krizhevsky. 2014. One weird trick for parallelizing convolutional neural networks. *arXiv preprint arXiv:1404.5997* (2014).
- [57] Prabhat Kumar, Govind P. Gupta, and Rakesh Tripathi. 2021. PEFL: Deep privacy-encoding based federated learning framework for smart agriculture. *IEEE Micro*. (2021).
- [58] Anusha Lalitha, Shubhanshu Shekhar, Tara Javidi, and Farinaz Koushanfar. 2018. Fully decentralized federated learning. In *Third Workshop on Bayesian Deep Learning (NeurIPS)*.
- [59] Jin-woo Lee, Jaehoon Oh, Sungsu Lim, Se-Young Yun, and Jae-Gil Lee. 2020. TornadoAggregate: Accurate and scalable federated learning via the ring-based architecture. *arXiv preprint arXiv:2012.03214* (2020).
- [60] Mo Li, Zhenjiang Li, and Athanasios V. Vasilakos. 2013. A survey on topology control in wireless sensor networks: Taxonomy, comparative study, and open issues. *Proc. IEEE* 101, 12 (2013), 2538–2557.
- [61] Qinbin Li, Zeyi Wen, Zhaomin Wu, Sixu Hu, Naibo Wang, Yuan Li, Xu Liu, and Bingsheng He. 2021. A survey on federated learning systems: Vision, hype and reality for data privacy and protection. *IEEE Transactions on Knowledge and Data Engineering* (2021).
- [62] Tian Li, Shengyuan Hu, Ahmad Beirami, and Virginia Smith. 2021. Ditto: Fair and robust federated learning through personalization. In *International Conference on Machine Learning*. PMLR, 6357–6368.
- [63] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, and Virginia Smith. 2020. Federated learning: Challenges, methods, and future directions. *IEEE Signal Processing Magazine* 37, 3 (2020), 50–60.
- [64] Xiang Li, Kaixuan Huang, Wenhao Yang, Shusen Wang, and Zhihua Zhang. 2019. On the convergence of FedAvg on non-IID data. *arXiv preprint arXiv:1907.02189* (2019).
- [65] Yuzheng Li, Chuan Chen, Nan Liu, Huawei Huang, Zibin Zheng, and Qiang Yan. 2020. A blockchain-based decentralized federated learning framework with committee consensus. *IEEE Network* 35, 1 (2020), 234–241.
- [66] Zexi Li, Jiaxun Lu, Shuang Luo, Didi Zhu, Yunfeng Shao, Yinchuan Li, Zhimeng Zhang, and Chao Wu. 2022. Mining latent relationships among clients: Peer-to-peer federated learning with adaptive neighbor matching. *arXiv preprint arXiv:2203.12285* (2022).

- [67] Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu. 2017. Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent. *arXiv preprint arXiv:1705.09056* (2017).
- [68] Wei Yang Bryan Lim, Nguyen Cong Luong, Dinh Thai Hoang, Yutao Jiao, Ying-Chang Liang, Qiang Yang, Dusit Niyato, and Chunyan Miao. 2020. Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 22, 3 (2020), 2031–2063.
- [69] Frank Po-Chen Lin, Seyyedali Hosseinalipour, Sheikh Shams Azam, Christopher G. Brinton, and Nicolo Michelusi. 2021. Semi-decentralized federated learning with cooperative D2D local model aggregations. *IEEE Journal on Selected Areas in Communications* (2021).
- [70] Fang Liu, Guoming Tang, Youhuizi Li, Zhiping Cai, Xingzhou Zhang, and Tongqing Zhou. 2019. A survey on edge computing systems and tools. *Proc. IEEE* 107, 8 (2019), 1537–1562.
- [71] Lumin Liu, Jun Zhang, S. H. Song, and Khaled B. Letaief. 2020. Client-edge-cloud hierarchical federated learning. In *ICC 2020-2020 IEEE International Conference on Communications (ICC)*. IEEE, 1–6.
- [72] Shaoshan Liu, Liangkai Liu, Jie Tang, Bo Yu, Yifan Wang, and Weisong Shi. 2019. Edge computing for autonomous driving: Opportunities and challenges. *Proc. IEEE* 107, 8 (2019), 1697–1716.
- [73] Wei Liu, Li Chen, and Wenyi Zhang. 2021. Decentralized federated learning: Balancing communication and computing costs. *arXiv preprint arXiv:2107.12048* (2021).
- [74] Yang Liu, Yan Kang, Chaoping Xing, Tianjian Chen, and Qiang Yang. 2020. A secure federated transfer learning framework. *IEEE Intelligent Systems* 35, 4 (2020), 70–82.
- [75] Yang Liu, Yan Kang, Xinwei Zhang, Liping Li, Yong Cheng, Tianjian Chen, Mingyi Hong, and Qiang Yang. 2019. A communication efficient collaborative learning framework for distributed features. *arXiv preprint arXiv:1912.11187* (2019).
- [76] Yunlong Lu, Xiaohong Huang, Yueyue Dai, Sabita Maharjan, and Yan Zhang. 2019. Blockchain and federated learning for privacy-preserved data sharing in industrial IoT. *IEEE Transactions on Industrial Informatics* 16, 6 (2019), 4177–4186.
- [77] Yunlong Lu, Xiaohong Huang, Yueyue Dai, Sabita Maharjan, and Yan Zhang. 2020. Federated learning for data privacy preservation in vehicular cyber-physical systems. *IEEE Network* 34, 3 (2020), 50–56.
- [78] Siqi Luo, Xu Chen, Qiong Wu, Zhi Zhou, and Shuai Yu. 2020. HFEL: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning. *IEEE Transactions on Wireless Communications* 19, 10 (2020), 6535–6548.
- [79] Qianpiao Ma, Yang Xu, Hongli Xu, Zhida Jiang, Liusheng Huang, and He Huang. 2021. FedSA: A semi-asynchronous federated learning mechanism in heterogeneous edge computing. *IEEE Journal on Selected Areas in Communications* 39, 12 (2021), 3654–3672.
- [80] Umer Majeed and Choong Seon Hong. 2019. FLchain: Federated learning via MEC-enabled blockchain network. In *2019 20th Asia-Pacific Network Operations and Management Symposium (APNOMS)*. IEEE, 1–4.
- [81] Yuyi Mao, Changsheng You, Jun Zhang, Kaibin Huang, and Khaled B. Letaief. 2017. A survey on mobile edge computing: The communication perspective. *IEEE Communications Surveys & Tutorials* 19, 4 (2017), 2322–2358.
- [82] Othmane Marfoq, Chuan Xu, Giovanni Neglia, and Richard Vidal. 2020. Throughput-optimal topology design for cross-silo federated learning. *Advances in Neural Information Processing Systems* 33 (2020), 19478–19487.
- [83] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agueray. Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial Intelligence and Statistics*. PMLR, 1273–1282.
- [84] Naram Mhaisen, Alaa Awad, Amr Mohamed, Aiman Erbad, and Mohsen Guizani. 2021. Optimal user-edge assignment in hierarchical federated learning based on statistical properties and network topology constraints. *IEEE Transactions on Network Science and Engineering* (2021).
- [85] Jed Mills, Jia Hu, and Geyong Min. 2019. Communication-efficient federated learning for wireless edge intelligence in IoT. *IEEE Internet of Things Journal* 7, 7 (2019), 5986–5994.
- [86] David Moher, Alessandro Liberati, Jennifer Tetzlaff, Douglas G. Altman, and Prisma Group. 2010. Preferred reporting items for systematic reviews and meta-analyses: The PRISMA statement. *International Journal of Surgery* 8, 5 (2010), 336–341.
- [87] Alberto Montresor and Márk Jelasity. 2009. PeerSim: A scalable P2P simulator. In *Proc. of the 9th Int. Conference on Peer-to-Peer (P2P'09)*. Seattle, WA, 99–100.
- [88] Virraji Mothukuri, Reza M. Parizi, Seyedamin Pouriyeh, Yan Huang, Ali Dehghantanha, and Gautam Srivastava. 2021. A survey on security and privacy of federated learning. *Future Generation Computer Systems* 115 (2021), 619–640.
- [89] Anh Nguyen, Tuong Do, Minh Tran, Binh X. Nguyen, Chien Duong, Tu Phan, Erman Tjiputra, and Quang D. Tran. 2021. Deep federated learning for autonomous driving. *arXiv preprint arXiv:2110.05754* (2021).

- [90] Dinh C. Nguyen, Ming Ding, Pubudu N. Pathirana, Aruna Seneviratne, Jun Li, and H. Vincent Poor. 2021. Federated learning for internet of things: A comprehensive survey. *IEEE Communications Surveys & Tutorials* 23, 3 (2021), 1622–1658.
- [91] Dinh C. Nguyen, Quoc-Viet Pham, Pubudu N. Pathirana, Ming Ding, Aruna Seneviratne, Zihuai Lin, Octavia Dobre, and Won-Joo Hwang. 2022. Federated learning for smart healthcare: A survey. *ACM Computing Surveys (CSUR)* 55, 3 (2022), 1–37.
- [92] John Nguyen, Kshitiz Malik, Maziar Sanjabi, and Michael Rabbat. 2022. Where to begin? Exploring the impact of pre-training and initialization in federated learning. *arXiv preprint arXiv:2206.15387* (2022).
- [93] Wanli Ni, Yuanwei Liu, Yonina C. Eldar, Zhaohui Yang, and Hui Tian. 2022. STAR-RIS integrated nonorthogonal multiple access and over-the-air federated learning: Framework, analysis, and optimization. *IEEE Internet of Things Journal* 9, 18 (2022), 17136–17156.
- [94] Huansheng Ning, Yunfei Li, Feifei Shi, and Laurence T. Yang. 2020. Heterogeneous edge computing open platforms and tools for internet of things. *Future Generation Computer Systems* 106 (2020), 67–76.
- [95] Róbert Ormándi, István Hegedűs, and Márk Jelasity. 2013. Gossip learning with linear models on fully distributed data. *Concurrency and Computation: Practice and Experience* 25, 4 (2013), 556–571.
- [96] Jianli Pan and James McElhannon. 2017. Future edge cloud and edge computing for internet of things applications. *IEEE Internet of Things Journal* 5, 1 (2017), 439–449.
- [97] Bjarne Pfitzner, Nico Steckhan, and Bert Arnrich. 2021. Federated learning in a medical context: A systematic literature review. *ACM Transactions on Internet Technology (TOIT)* 21, 2 (2021), 1–31.
- [98] Krishna Pillutla, Kshitiz Malik, Abdel-Rahman Mohamed, Mike Rabbat, Maziar Sanjabi, and Lin Xiao. 2022. Federated learning with partial model personalization. In *International Conference on Machine Learning*. PMLR, 17716–17758.
- [99] Pinyarash Pinyoanuntapong, Prabhu Janakaraj, Pu Wang, Minwoo Lee, and Chen Chen. 2020. FedAir: Towards multi-hop federated learning over-the-air. In *2020 IEEE 21st International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*. IEEE, 1–5.
- [100] Shiva Raj Pokhrel and Jinho Choi. 2020. Federated learning with blockchain for autonomous vehicles: Analysis and design challenges. *IEEE Transactions on Communications* 68, 8 (2020), 4734–4746.
- [101] Julien Polge, Jérémy Robert, and Yves Le Traon. 2021. Permissioned blockchain frameworks in the industry: A comparison. *ICT Express* 7, 2 (2021), 229–233.
- [102] Tie Qiu, Jiancheng Chi, Xiaobo Zhou, Zhaolong Ning, Mohammed Atiquzzaman, and Dapeng Oliver Wu. 2020. Edge computing in industrial internet of things: Architecture, advances and challenges. *IEEE Communications Surveys & Tutorials* 22, 4 (2020), 2462–2488.
- [103] Youyang Qu, Longxiang Gao, Tom H. Luan, Yong Xiang, Shui Yu, Bai Li, and Gavin Zheng. 2020. Decentralized privacy using blockchain-enabled federated learning in fog computing. *IEEE Internet of Things Journal* 7, 6 (2020), 5171–5183.
- [104] Zhaonan Qu, Kaixiang Lin, Zhaojian Li, Jiayu Zhou, and Zhengyuan Zhou. 2020. A unified linear speedup analysis of stochastic FedAvg and Nesterov accelerated FedAvg. *arXiv e-prints* (2020), arXiv–2007.
- [105] Rajmohan Rajaraman. 2002. Topology control and routing in ad hoc networks: A survey. *ACM SIGACT News* 33, 2 (2002), 60–73.
- [106] Amirhossein Reiszadeh, Aryan Mokhtari, Hamed Hassani, Ali Jadbabaie, and Ramtin Pedarsani. 2020. FedPAQ: A communication-efficient federated learning method with periodic averaging and quantization. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 2021–2031.
- [107] Nicola Rieke, Jonny Hancox, Wenqi Li, Fausto Milletari, Holger R. Roth, Shadi Albarqouni, Spyridon Bakas, Mathieu N. Galtier, Bennett A. Landman, Klaus Maier-Hein, Sébastien Ourselin, Micah Sheller, Ronald M. Summers, Andrew Trask, Daguang Xu, Maximilian Baust, and M. Jorge Cardoso. 2020. The future of digital health with federated learning. *NPJ Digital Medicine* 3, 1 (2020), 1–7.
- [108] Abhijit Guha Roy, Shayan Siddiqui, Sebastian Pölsterl, Nassir Navab, and Christian Wachinger. 2019. BrainTorrent: A peer-to-peer environment for decentralized federated learning. *arXiv preprint arXiv:1905.06731* (2019).
- [109] Yichen Ruan, Xiaoxi Zhang, Shu-Che Liang, and Carlee Joe-Wong. 2021. Towards flexible device participation in federated learning. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 3403–3411.
- [110] Alessio Sacco, Flavio Esposito, and Guido Marchetto. 2020. A federated learning approach to routing in challenged SDN-enabled edge networks. In *2020 6th IEEE Conference on Network Softwarization (NetSoft)*. IEEE, 150–154.
- [111] Anit Kumar Sahu, Tian Li, Maziar Sanjabi, Manzil Zaheer, Ameet Talwalkar, and Virginia Smith. 2018. On the convergence of federated optimization in heterogeneous networks. *arXiv preprint arXiv:1812.06127* 3 (2018), 3.
- [112] Mahadev Satyanarayanan. 2017. The emergence of edge computing. *Computer* 50, 1 (2017), 30–39.
- [113] Stefano Savazzi, Monica Nicoli, and Vittorio Rampa. 2020. Federated learning with cooperating devices: A consensus approach for massive IoT networks. *IEEE Internet of Things Journal* 7, 5 (2020), 4641–4654.
- [114] Muhammad Shayan, Clement Fung, Chris J. M. Yoon, and Ivan Beschastnikh. 2020. Biscotti: A blockchain system for private and secure federated learning. *IEEE Transactions on Parallel and Distributed Systems* 32, 7 (2020), 1513–1525.

- [115] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. Edge computing: Vision and challenges. *IEEE Internet of Things Journal* 3, 5 (2016), 637–646.
- [116] Weisong Shi and Schahram Dustdar. 2016. The promise of edge computing. *Computer* 49, 5 (2016), 78–81.
- [117] Yi Shi, Yalin E. Sagduyu, and Tugba Erpek. 2022. Federated learning for distributed spectrum sensing in NextG communication networks. *arXiv preprint arXiv:2204.03027* (2022).
- [118] Yandong Shi, Yong Zhou, and Yuanming Shi. 2021. Over-the-air decentralized federated learning. *arXiv preprint arXiv:2106.08011* (2021).
- [119] Yushan Siriwardhana, Pawani Porambage, Madhusanka Liyanage, and Mika Ylianttila. 2021. A survey on mobile augmented reality with 5G mobile edge computing: Architectures, applications, and technical aspects. *IEEE Communications Surveys & Tutorials* 23, 2 (2021), 1160–1192.
- [120] Samuel L. Smith, Pieter-Jan Kindermans, Chris Ying, and Quoc V. Le. 2017. Don't decay the learning rate, increase the batch size. *arXiv preprint arXiv:1711.00489* (2017).
- [121] Dimitris Stripelis and José Luis Ambite. 2021. Semi-synchronous federated learning. *arXiv preprint arXiv:2102.02849* (2021).
- [122] Haijian Sun, Fuhui Zhou, and Rose Qingyang Hu. 2019. Joint offloading and computation energy efficiency maximization in a mobile edge computing system. *IEEE Transactions on Vehicular Technology* 68, 3 (2019), 3052–3056.
- [123] Canh T. Dinh, Nguyen Tran, and Josh Nguyen. 2020. Personalized federated learning with Moreau envelopes. *Advances in Neural Information Processing Systems* 33 (2020), 21394–21405.
- [124] Alysa Ziyang Tan, Han Yu, Lizhen Cui, and Qiang Yang. 2022. Towards personalized federated learning. *IEEE Transactions on Neural Networks and Learning Systems* (2022).
- [125] Yangyang Tao, Junxiu Zhou, and Shucheng Yu. 2021. Efficient parameter aggregation in federated learning with hybrid convergecast. In *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*. IEEE, 1–6.
- [126] Luke K. Topham, Wasif Khan, Dhiya Al-Jumeily, Atif Waraich, and Abir J. Hussain. 2022. Gait identification using limb joint movement and deep machine learning. *IEEE Access* 10 (2022), 100113–100127.
- [127] Tuyen X. Tran and Dario Pompili. 2018. Joint task offloading and resource allocation for multi-server mobile-edge computing networks. *IEEE Transactions on Vehicular Technology* 68, 1 (2018), 856–868.
- [128] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. 2018. Split learning for health: Distributed deep learning without sharing raw patient data. *arXiv preprint arXiv:1812.00564* (2018).
- [129] Paul Voigt and Axel Von dem Bussche. 2017. The EU general data protection regulation (GDPR). *A Practical Guide, 1st Ed., Cham: Springer International Publishing* 10, 3152676 (2017), 10–5555.
- [130] Aidmar Wainakh, Alejandro Sanchez Guinea, Tim Grube, and Max Mühlhäuser. 2020. Enhancing privacy via hierarchical federated learning. In *2020 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 344–347.
- [131] Haoxin Wang, Tingting Liu, Baekgyu Kim, Chung-Wei Lin, Shinichi Shiraishi, Jiang Xie, and Zhu Han. 2020. Architectural design alternatives based on cloud/edge/fog computing for connected vehicles. *IEEE Communications Surveys & Tutorials* 22, 4 (2020), 2349–2377.
- [132] Su Wang, Mengyuan Lee, Seyyedali Hosseinalipour, Roberto Morabito, Mung Chiang, and Christopher G. Brinton. 2021. Device sampling for heterogeneous federated learning: Theory, algorithms, and implementation. *arXiv preprint arXiv:2101.00787* (2021).
- [133] Shangguang Wang, Yali Zhao, Jinlinag Xu, Jie Yuan, and Ching-Hsien Hsu. 2019. Edge server placement in mobile edge computing. *J. Parallel and Distrib. Comput.* 127 (2019), 160–168.
- [134] Tian Wang, Yucheng Lu, Jianhuang Wang, Hong-Ning Dai, Xi Zheng, and Weijia Jia. 2021. EIHPD: Edge-intelligent hierarchical dynamic pricing based on cloud-edge-client collaboration for IoT systems. *IEEE Trans. Comput.* 70, 8 (2021), 1285–1298.
- [135] Xiaofei Wang, Yiwen Han, Chenyang Wang, Qiyang Zhao, Xu Chen, and Min Chen. 2019. In-edge AI: Intelligentizing mobile edge computing, caching and communication by federated learning. *IEEE Network* 33, 5 (2019), 156–165.
- [136] Xiaokang Wang, Laurence T. Yang, Xia Xie, Jirong Jin, and M. Jamal Deen. 2017. A cloud-edge computing framework for cyber-physical-social services. *IEEE Communications Magazine* 55, 11 (2017), 80–85.
- [137] Zhao Wang, Yifan Hu, Jun Xiao, and Chao Wu. 2021. Efficient ring-topology decentralized federated learning with deep generative models for industrial artificial intelligent. *arXiv preprint arXiv:2104.08100* (2021).
- [138] Zhiyuan Wang, Hongli Xu, Jianchun Liu, Yang Xu, He Huang, and Yangming Zhao. 2022. Accelerating federated learning with cluster construction and hierarchical aggregation. *IEEE Transactions on Mobile Computing* (2022).
- [139] Kang Wei, Jun Li, Chuan Ma, Ming Ding, Sha Wei, Fan Wu, Guihai Chen, and Thilina Ranbaduge. 2022. Vertical federated learning: Challenges, methodologies and experiments. *arXiv preprint arXiv:2202.04309* (2022).
- [140] Wanli Wen, Zihan Chen, Howard H. Yang, Wenchao Xia, and Tony Q. S. Quek. 2022. Joint scheduling and resource allocation for hierarchical federated edge learning. *IEEE Transactions on Wireless Communications* (2022).

- [141] Cong Xie, Sanmi Koyejo, and Indranil Gupta. 2019. Asynchronous federated optimization. *arXiv preprint arXiv:1903.03934* (2019).
- [142] Hong Xing, Osvaldo Simeone, and Suzhi Bi. 2020. Decentralized federated learning via SGD over wireless D2D networks. In *2020 IEEE 21st International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*. IEEE, 1–5.
- [143] Xiaolong Xu, Qingxiang Liu, Yun Luo, Kai Peng, Xuyun Zhang, Shunmei Meng, and Lianyong Qi. 2019. A computation offloading method over big data for IoT-enabled cloud-edge computing. *Future Generation Computer Systems* 95 (2019), 522–533.
- [144] Guang Yang, Ke Mu, Chunhe Song, Zhijia Yang, and Tierui Gong. 2021. RingFed: Reducing communication costs in federated learning on non-IID data. *arXiv preprint arXiv:2107.08873* (2021).
- [145] Kai Yang, Tao Jiang, Yuanming Shi, and Zhi Ding. 2020. Federated learning via over-the-air computation. *IEEE Transactions on Wireless Communications* 19, 3 (2020), 2022–2035.
- [146] Qiang Yang, Yang Liu, Tianjian Chen, and Yongxin Tong. 2019. Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)* 10, 2 (2019), 1–19.
- [147] Yunfan Ye, Shen Li, Fang Liu, Yonghao Tang, and Wanting Hu. 2020. EdgeFed: Optimized federated learning based on edge computing. *IEEE Access* 8 (2020), 209191–209198.
- [148] Michal Yemini, Rajarshi Saha, Emre Ozfatura, Deniz Gündüz, and Andrea J. Goldsmith. 2022. Robust federated learning with connectivity failures: A semi-decentralized framework with collaborative relaying. *arXiv preprint arXiv:2202.11850* (2022).
- [149] Rong Yu and Peichun Li. 2021. Toward resource-efficient federated learning in mobile edge computing. *IEEE Network* 35, 1 (2021), 148–155.
- [150] Wei Yu, Fan Liang, Xiaofei He, William Grant Hatcher, Chao Lu, Jie Lin, and Xinyu Yang. 2017. A survey on the edge computing for the Internet of Things. *IEEE Access* 6 (2017), 6900–6919.
- [151] Jinliang Yuan, Mengwei Xu, Xiao Ma, Ao Zhou, Xuanzhe Liu, and Shangguang Wang. 2020. Hierarchical federated learning through LAN-WAN orchestration. *arXiv preprint arXiv:2010.11612* (2020).
- [152] Shahryar Zehtabi, Seyyedali Hosseinalipour, and Christopher G. Brinton. 2022. Decentralized event-triggered federated learning with heterogeneous communication thresholds. *arXiv preprint arXiv:2204.03726* (2022).
- [153] Chong Zhang, Xiao Liu, Xi Zheng, Rui Li, and Huai Liu. 2020. FengHuoLun: A federated learning based edge computing platform for cyber-physical systems. In *2020 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. IEEE, 1–4.
- [154] Chen Zhang, Yu Xie, Hang Bai, Bin Yu, Weihong Li, and Yuan Gao. 2021. A survey on federated learning. *Knowledge-Based Systems* 216 (2021), 106775.
- [155] Jie Zhang, Xiaohua Qi, and Bo Zhao. 2023. Federated generative learning with foundation models. *arXiv preprint arXiv:2306.16064* (2023).
- [156] Jing Zhang, Weiwei Xia, Feng Yan, and Lianfeng Shen. 2018. Joint computation offloading and resource allocation optimization in heterogeneous networks with mobile edge computing. *IEEE Access* 6 (2018), 19324–19337.
- [157] Ke Zhang, Yuming Mao, Supeng Leng, Quanxin Zhao, Longjiang Li, Xin Peng, Li Pan, Sabita Maharjan, and Yan Zhang. 2016. Energy-efficient offloading for mobile edge computing in 5G heterogeneous networks. *IEEE Access* 4 (2016), 5896–5907.
- [158] Weishan Zhang, Qinghua Lu, Qiuyu Yu, Zhaotong Li, Yue Liu, Sin Kit Lo, Shiping Chen, Xiwei Xu, and Liming Zhu. 2020. Blockchain-based federated learning for device failure detection in industrial IoT. *IEEE Internet of Things Journal* 8, 7 (2020), 5926–5937.
- [159] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. 2018. Federated learning with non-IID data. *arXiv preprint arXiv:1806.00582* (2018).
- [160] Yang Zhao, Jun Zhao, Linshan Jiang, Rui Tan, Dusit Niyato, Zengxiang Li, Lingjuan Lyu, and Yingbo Liu. 2020. Privacy-preserving blockchain-based federated learning for IoT devices. *IEEE Internet of Things Journal* 8, 3 (2020), 1817–1829.
- [161] Zhicong Zhong, Yipeng Zhou, Di Wu, Xu Chen, Min Chen, Chao Li, and Quan Z. Sheng. 2021. P-FedAvg: Parallelizing federated learning with theoretical guarantees. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [162] Chunyi Zhou, Anmin Fu, Shui Yu, Wei Yang, Huaqun Wang, and Yuqing Zhang. 2020. Privacy-preserving federated learning in fog computing. *IEEE Internet of Things Journal* 7, 11 (2020), 10782–10793.
- [163] Xiaokang Zhou, Wei Liang, Jinhua She, Zheng Yan, I. Kevin, and Kai Wang. 2021. Two-layer federated learning with heterogeneous model aggregation for 6G supported internet of vehicles. *IEEE Transactions on Vehicular Technology* 70, 6 (2021), 5308–5317.

- [164] Bingzhao Zhu, Xingjian Shi, Nick Erickson, Mu Li, George Karypis, and Mahsa Shoaran. 2023. XTab: Cross-table pretraining for tabular transformers. *arXiv preprint arXiv:2305.06090* (2023).
- [165] Juncen Zhu, Jiannong Cao, Divya Saxena, Shan Jiang, and Houda Ferradi. 2023. Blockchain-empowered federated learning: Challenges, solutions, and future directions. *Comput. Surveys* 55, 11 (2023), 1–31.

Received 6 February 2023; revised 1 April 2024; accepted 4 April 2024